Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ7 «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ НА ТЕМУ:

Система управления корпусными данными параллельных корпусов технических текстов

Студент **ИУ7-74Б** _____ **И. С. Бугаков**
(Подпись, дата) (И.О.Фамилия)

Руководитель _____ **Ю. В. Строганов**
(Подпись, дата) (И.О.Фамилия)

Консультант _____ **Ю. И. Бутенко**
(Подпись, дата) (И.О.Фамилия)

Рекомендованная руководителем НИР оценка: _____

2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой ИУ7
_____ И. В. Рудаков
«__» сентября 2025 г.

ЗАДАНИЕ
на выполнение научно-исследовательской работы

по теме

Система управления корпусными данными параллельных корпусов технических текстов

Студент группы **ИУ7-74Б**

Бугаков Иван Сергеевич

Направленность НИР (учебная, исследовательская, практическая, производственная, др.): учебная.

Источник тематики (кафедра, предприятие, НИР): кафедра.

График выполнения НИР: 25% к 5 нед., 50% к 8 нед., 75% к 11 нед., 100% к 15 нед.

Техническое задание

Проанализировать предметную область использования параллельных корпусов в корпусной лингвистике. Привести классификацию корпусов текстов. Перечислить виды выравнивая и разметки текстов в параллельном корпусе. Перечислить методы построения разметки текстов в параллельном корпусе. Сформулировать критерии и сравнить перечисленные методы построения разметки применительно к параллельным корпусам технических текстов. Проанализировать и сформулировать критерии сравнения существующих систем управления корпусными данными. По сформулированным критериям провести сравнение существующих систем управления корпусными данными.

Оформление научно-исследовательской работы:

Расчетно-пояснительная записка на 12-30 листах формата А4.

Перечень графического (иллюстративного) материала:

Презентация на 3-8 слайдах.

Дата выдачи задания «__» сентября 2025 г.

Руководитель НИР _____ **Ю. В. Строганов**

Студент _____ **И. С. Бугаков**

СОДЕРЖАНИЕ

РЕФЕРАТ **4**

1 Анализ предметной области **5**

1.1 Основные понятия корпусной лингвистики 5

1.2 История развития корпусной лингвистики 6

1.3 Классификация корпусов 7

1.4 Репрезентативность корпуса 10

1.5 Виды разметок 10

2 Методы построения разметок текстов **14**

2.1 Анафорическая разметка 14

2.1.1 Вычисление семантической близости 14

2.1.2 ABBYY Compreno 15

2.1.3 CogNIAC 16

2.1.4 Метод опорных векторов 17

2.1.5 Дерево принятия решений 19

2.2 Морфологическая разметка 21

2.2.1 Finite state transducer (FST) 21

2.2.2 Словарные подходы с использованием эвристик 23

2.2.3 Статистические подходы 27

2.3 Синтаксическая разметка 31

2.3.1 Дерево составляющих 32

2.3.2 Дерево зависимостей 33

3 Сравнение методов построения разметки **36**

3.1 Анафорическая разметка 36

3.1.1 Критерии и сравнение методов 36

3.1.2 Выводы 38

3.2 Морфологическая разметка 38

3.2.1 Критерии и сравнение методов 38

3.2.2 Выводы 40

3.3 Синтаксическая разметка 40
3.3.1 Критерии и сравнение методов 40
3.3.2 Выводы 42

4 Сравнение корпусных менеджеров 43
4.1 Существующие корпусные менеджеры 43
4.2 Критерии сравнения корпусных менеджеров 43
4.3 Выводы 45

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 46

РЕФЕРАТ

Расчетно-пояснительная записка 54 с., 8 рис., 10 таблиц, 54 источника, 1 приложение.

1 Анализ предметной области

1.1 Основные понятия корпусной лингвистики

Корпусная лингвистика — раздел компьютерной лингвистики, занимающийся разработкой общих принципов построения и использования лингвистических корпусов (корпусов текстов) с применением компьютерных технологий. Под лингвистическим, или языковым, корпусом текстов понимается большой, представленный в машиночитаемом виде, унифицированный, структурированный, размеченный, филологически компетентный массив языковых данных, предназначенный для решения конкретных лингвистических задач [1]. Существуют также альтернативные определения понятия «языковой корпус». Например, корпус текстов – вид корпуса данных, единицами которого являются тексты или их достаточно значительные фрагменты, включающие, например, какие-то полные фрагменты макроструктуры текстов определённой проблемной области [2].

В современной компьютерной лингвистике понятие «корпус текстов» подразумевает также специализированную поисковую систему, включающую программные средства для поиска данных в корпусе, получения статистической информации и предоставления пользователю результатов в удобной форме, называемую корпусным менеджером [1].

В соответствии с описанным выше, современный корпус предполагает также наличие аннотации или разметки. Аннотация — это приписанная всем единицам выбранного уровня (текст, предложение, словоформа и т. д.) соответствующая лингвистическая информация. При этом аннотирование может быть представлено одновременно для различных уровней [3].

Поиск по слову в корпусе позволяет построить конкорданс — список всех словоупотреблений данного слова в контексте с указанием источников [1]. Таким образом корпусы могут быть использованы для получения разнообразных справок и статистических данных о языковых я речевых единицах. В более широком смысле конкорданс — особый тип словаря, в котором каждое слово или понятие сопровождается минимальным контекстом и всеми случаями употребления в некотором тексте. Иногда конкроданс также определяют как словарь сочетаемости языковых единиц, словарь контекстов [4]. Конкордансы могут быть использованы в лингвистике в следующих целях [5]:

— сравнение различных использований одного слова;
— анализ частотности слов и словосочетаний;
— поиск и исследование фраз и идиом;
— поиск перевода, например, терминологии;
— анализ ключевых слов.

Конкордансы также могут использоваться для анализа творчества отдельных писателей [6, 7] и обучения иностранным языкам [8].

Тезаурус в современной лингвистике — особая разновидность словарей общей или специ-

альной лексики, в которых указаны семантические отношения (синонимы, антонимы, паронимы, гипонимы, гиперонимы и т. п.) между лексическими единицами [3].

Еще одним понятием корпусной лингвистики является онтология. Несмотря на множество определений онтологии многие авторы соглашаются в наборе основных компонентов онтологии: классы или понятия; атрибуты (свойства); экземпляры (отдельные индивиды), отношения между классами или экземплярами; аксиомы онтологии. Исходя из этого формально онтология определяется соотношением 1.1.

$$O =< C,\ E,\ A_t,\ R,\ A >, \tag{1.1}$$

где C — понятия (классы) онтологии,
E — экземпляры онтологии,
A_t — атрибуты понятий и экземпляров онтологии,
R — отношения между понятиями,
A — аксиомы онтологии.

Таким образом объектом корпусной лингвистики является корпус текстов, который, с одной стороны, представляет собой исходный материал для корпусной лингвистики и других разделов лингвистики; с другой стороны, является результатом деятельности корпусной лингвистики [1].

Предметом корпусной лингвистики являются теоретические основы и практические механизмы создания и использования представительных массивов языковых данных, предназначенных для лингвистических исследований в интересах широкого круга пользователей [1].

1.2 История развития корпусной лингвистики

Истоки корпусной лингвистики, ее приемов и методов прослеживаются начиная с момента создания грамматики санскрита Панини около V в. до н. э., представлявшей собой систематизацию уже существовавшей в ритуальных текстах грамматики [3, 9].

В дальнейшем основным направлением развития корпусной лингвистики стало составление конкордансов основанных на Библии. Первым из известных на данный момент конкордансов является связанный с именем Антония Падуанского «Concordantiae morales sacrae scripturae» («Нравственная конкорданция Священного Писания»), появившийся в начале XIII века [3, 10].

Следующим этапом развития корпусной лингвистики связан с развитием лексикографии и создания словарей языков в XVIII–XIX веках. Появившиеся в это время словари и грамматики составлялись на основе крупных картотек или произведений признанных авторов [3]. Так, например, Словарь живого великорусского языка В. И. Даля основан на материалах как предшествующих ему словарей, так и на самостоятельно подобранных, при этом слова приводятся с примерами использования, что близко по смыслу к конкордансу, построенному на основе существовавших на тот момент литературного и народного языков [3, 11]. В английском языке

словарями, основанными на обработке и выборке примеров употребления из существующих произведений литературы, принято считать «Американский словарь английского языка» («An American dictionary of the English language») Н. Вебстера [12] и толковый словарь английского языка («Dictionary of the English language») С. Джонсона [13]. Г. Пауль для составления немецкой грамматики («Deutsche Grammatik» [14]) использовал произведения немецких «классиков» для иллюстрации своих утверждений [1].

В конце XIX – начале XX века начинают появляться корпусы специально созданные для лингвистических исследований или решения практических задач. Одной из таких задач являлся подсчет частотности языковых единиц. Первым словарем такого рода стал «Частотный словарь немецкого языка» («Häufigkeitswörterbuch der deutschen Sprache» [15]). Данный словарь был подготовлен на основе корпуса содержащего одиннадцать миллионов словоупотреблений [3].

В 1909 году был опубликован первый том «A Modern English Grammar on Historical Principles. Part I. Sounds and spellings» [16] датского ученого Отто Есперсена. Основной идеей труда Есперсена был переход от нормативных грамматик к дескриптивным, т. е. анализирующим и описывающим использование языка в живой речи. Для подкрепления изложенных в грамматике тезисов Есперсен специально подбирал источники примеров, общее изложение которых заняло 40 страниц и представляет собой прообраз современного репрезентативного корпуса [3]. В связи с необходимостью ручного подбора и обработки выход всех семи томов данной работы затянулся на 40 лет.

1.3 Классификация корпусов

В зависимости от цели разработки корпусы могут классифицироваться самым разным образом. Выделенные в [1] типы классификаций представлены в таблице 1.1.

Таблица 1.1 — Виды классификаций корпусов

Признак классификации	Типы корпусов
Тип языковых данных	Письменные
	Устные
	Смешанные
«Параллельность»	Одноязычные
	Двуязычные
	Многоязычные
«Литературность»	Литературные
	Диалектные
	Разговорные
	Терминологические
	Смешанные
Цель	Многоцелевые
	Специализированные
Жанр	Литературные
	Фольклорные
	Драматургические
	Публицистические
Доступность	Свободно доступные
	Коммерческие
	Закрытые
Назначение	Исследовательские
	Иллюстративные
Динамичность	Динамические (мониторные)
	Статические
Разметка	Размеченные
	Неразмеченные
Объем текстов	Полнотекстовые
	«Фрагментнотекстовые»

По типу языковых данных корпусы делятся на письменные, устные, смешанные. В письменных корпусах (Брауновский корпус, корпус Ланкастер-Осло-Берген) устная речь не представлена, в устных представлена только устная, к смешанным обычно относят национальные корпусы, представляющие язык в определенный период времени (Национальный корпус русского языка, Британский национальный корпус) [1].

По критерию параллельности корпусы делятся на одноязычные, двуязычные и многоязычные. В одноязычных противопоставляются диалекты, варианты языка. Двуязычные и многоязычные корпусы объединяют тексты одной и той же тематической области, независимо написанные на двух или нескольких языках. Зачастую основой таких корпусов также является множество текстов-оригиналов на каком-либо исходном языке и текстов-переводов исходных текстов на один или несколько других языков [1]. В [3] двуязычные относят к многоязычным, среди которых дополнительно выделяют выровненные (параллельные), т. е. связанные на уровне абзацев или предложений, и невыровненные. В случае использования параллельных корпусов построенных на основе текстов-переводов возможно автоматическое выравнивание на этапе

построения таких текстов [17, 18].

По критерию «литературности» выделяются литературные, диалектные, разговорные и терминологические корпусы [1].

По цели создания корпусы делятся на многоцелевые и специализированные. Многоцелевые корпусы содержат тексты различных жанров (к ним же относятся национальные корпусы), тогда как специализированные корпусы могут ограничиваться одним жанром или группой жанров [1].

Корпусы текстов могут быть классифицированы по жанрам и подразделяться на литературные, фольклорные, драматургические, публицистические и др. Примерами публицистического корпусы могут служить Компьютерный корпус текстов русских газет конца XX-ого века и корпус политических метафор.

По доступности корпусы подразделяются на свободно доступные, коммерческие и закрытые. Стоит отметить, что закрытые корпусы предназначены для специфических задач и потому не представлены в открытом доступе [1]. Иногда дополнительно выделяют также доступные по академической лицензии для использования в научной некоммерческой деятельности [3].

По назначению выделяют исследовательские и иллюстративные корпусы. Исследовательские корпусы создаются с целью с целью изучения различных аспектов функционирования языка. Поскольку этот тип корпусов ориентирован на широкий класс лингвистических задач, что требует при построении таких корпусов использовать специальные способы обеспечения репрезентативности. Такие корпусы содержат как правило от десятков до сотен миллионов словоупотреблений. Иллюстративные корпусы создаются после проведения исследования с целью подтверждения и обоснования полученных результатов [1].

Критерий «динамичность» подразделяет корпусы на динамические и статические. Статические корпусы содержат корпусы небольшого временного промежутка [18]. Типичными представителями такого рода корпусов являются авторские корпусы — коллекции текстов писателей [1]. Динамические корпусы служат для выявления функционирования языковых феноменов на временной шкале — например, изменения значения слов, частоты использования тех или иных синтаксических конструкций [17].

По критерию «разметка» корпусы делятся на размеченные и неразмеченные. В размеченном корпусе словам или предложениям присваиваются метки (тэги) в соответствии с характером разметки: морфологические, синтаксические, семантические, просодические и другие [1].

По критерию «объем текстов» выделяются полнотекстовые и так называемые фрагментотекстовые. Брауновский корпус и корпус Ланкастер-Осло-Берген были строго ограничены числом словоупотреблений, поэтому являются фрагментотекстовыми. К полнотекстовым относятся как правило корпусы определенного автора, а также корпусы коротких текстов, например, корпусы газетных заголовков [1].

В [2] выделяются дополнительно критерии общности и хронологического аспекта. По критерию «общности» корпусы разделяют на коллекции текстов принадлежащие одному писателю и различным. По критерию хронологического аспекта корпусы делят на синхронические

и диахронические, которые в сущности совпадают с ранее описанными динамическими и статическими типами.

Дополнительно среди фрагментнотекстовых корпусов выделяют конкорданcные и корпусы n-грамм [3].

Таким образом в данной работе рассматриваются письменные, выровненные, многоязычные, научно-технические, свободно доступные, исследовательские, статические, неразмеченные, фрагментнотекстовые корпусы.

1.4 Репрезентативность корпуса

Представительность корпуса получила название репрезентативности, а соотношение отдельных его частей (по разным характеристикам) — сбалансированности [19]. В настоящее время считается, что объем общеязыкового корпуса должен составлять не менее 100 миллионов словоупотреблений. Имеются разные походы к определению репрезентативности. Одним из способов является измерение насыщенности частотного словаря (относительные частоты лексем не меняются или меняются незначительно) [1].

Поскольку корпус представляет собой уменьшенную модель языка для специальных целей (то есть языка технических текстов), под его сбалансированностью понимается необходимое и достаточное пропорциональное представление в корпусе различных текстов данной проблемной области или данного типа, т. е. способность отражать все свойства изучаемого подъязыка [19].

Когда говорят о сбалансированном корпусе, то понимают также сбалансированность его по возможностям: правильно налаженный корпус предполагает модель программно-лингвистического инструмента, поддающуюся настройке и регулированию [19].

1.5 Виды разметок

Джеффри Лич приводит следующие принципы, которым должна подчиняться разметка общедоступного корпуса [20]:

— разметка должна основываться на доступной для пользователя в виде руководства или инструкции схеме анализа, в которой должно быть мотивированно введение каждого параметра;

— разметка общедоступного корпуса должна опираться на общеизвестную систему правил и классификаций;

— должно быть ясно кто и как разрабатывает схему аннотации и каковы ограничения, например юридические или технические, при использовании корпусом [3].

В соответствии с [1, 3, 21, 22] выделяются следующие типы разметки:

— анафорическая;

— дискурсная;
— жанровая;
— издательская;
— метатекстовая;
— морфологическая;
— синтаксическая;
— семантическая;
— служебная;
— структурная;
— терминологическая.

Анафорическая разметка необходима для установления референтных связей, т. е. соотнесение включенных в речь имен, именных выражений или их эквивалентов к реальным объектам. В связи с широким распространением в технических текстах аббревиатур и других референций вместо определенного термина (например, «аппарат», «ЛА» вместо «летательный аппарат»), при использовании средств машинной разметки таких текстов необходимо учесть все возможные референции встречающихся терминов, чтобы итоговая статистическая картина частотности употребления термина не была искаженной.

Дискурсная разметка зачастую реализуется для текстов художественного и разговорного стилей, речевых корпусов. Поскольку научно-техническим текстам не свойственно присутствие дискурсных единиц [23], реализация данной разметки для корпусов таких текстов не является необходимой.

Жанровая разметка для корпуса научно-технических текстов реализуется при отборе текстов для наполнения корпуса.

Издательская разметка отражает особенности форматирования текста, которые могут быть использованы при реализации других видов разметок.

Метатекстовая (экстралингвистическая) подразумевает под собой добавление к тексту внетекстовых характеристик: авторов, год, издательство, журнал, жанр и другие особенности текста или обстоятельства его создания. Выделяют два класса факторов влияющих на язык текстов и описываемых метатекстовой разметкой [1]:

— внешние, внеязыковые факторы (E — external);
— внутренние, относящие к внешним признакам текста (I — internal).

Дж. Синклер выделяет три группы E-факторов [24]:

— E1 (origin) — факторы, относящиеся к созданию текста автором;
— E2 (state) — факторы относящиеся к внешним признакам текста;
— E3 (aims) — факторы, относящиеся к причинам создания текста и его влиянию на аудиторию.

И две группы I-факторов:

— I1 (topic) — предметная область текста;
— I2 (style) — стилистические особенности текста (стиль, жанр).

Морфологическая разметка включает в себя: лемму (начальную форму слова), признак части речи, признаки грамматических категорий в зависимости от части речи. Данный тип разметки — основной и реализован в большинстве корпусов, а также является основой для дальнейших форм анализа — синтаксического и семантического. Кроме того в современных корпусах, этот тип разметки может быть реализован автоматически [1].

В корпусной лингвистике синтаксическая разметка основана на грамматике зависимостей [1, 3]. Тексты, получившие синтаксическую разметку известны в англоязычной литературе как treebanks [25]. Пример визуализации синтаксических зависимостей, отображаемых синтаксической разметкой, представлен на рисунке 1.1.

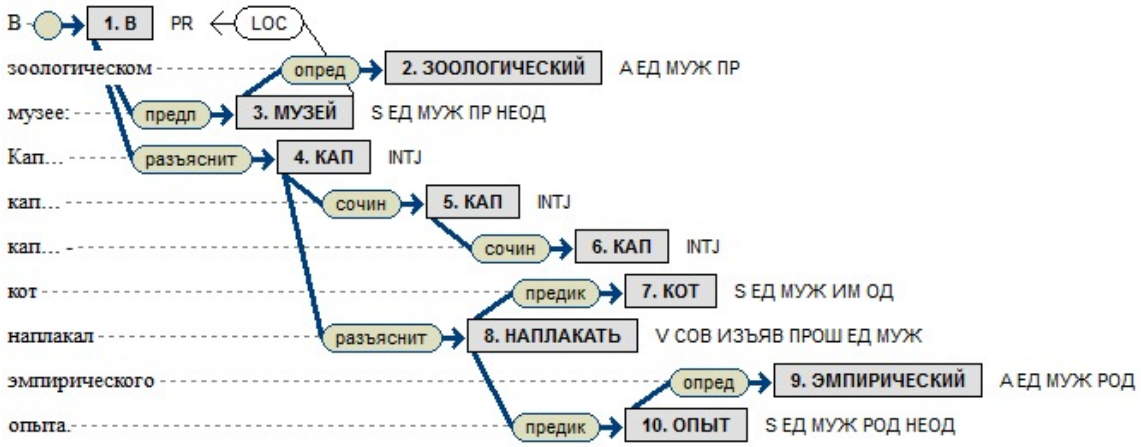

Рисунок 1.1 — Визуализация синтаксических зависимостей в НКРЯ

Семантическая разметка состоит в приписывании единицам текста одной или нескольких семантических категорий. Семантические тэги чаще всего обозначают семантические категории, к которым относится данное слово или словосочетание, и более узкие подкатегории, специфицирующие его значение. Семантическая разметка корпусов предусматривает определение значения слов, разрешение омонимии и категоризацию слов, выделение тематических классов и т. д. [1].

При этом к семантической разметке относят и категоризацию семантических ролей [3]. Семантические роли под названием «глубинные падежи» впервые выделены и описаны Ч. Филлмором [26]. Семантическая роль имени при данном предикате является частью семантики этого предиката и отражает общие свойства участников определенных групп ситуаций. В данном определении предполагается, что предикатные (глагольные) лексемы описывают «ситуации», а именные лексемы — «участников» этих ситуаций, которые называются аргументами предикатов. Состав и количество семантических ролей, выделяемых при том или ином описании, зависят от конкретной задачи и могут существенно различаться; в большинстве случаев достаточно сравнительно небольшого набора из двух - трех десятков ролей. [27].

Служебная разметка используется для приписывания текстам информации о стиле и целевой аудитории текстов в корпусе. Реализация этого типа разметки в параллельном корпусе технических текстов нецелесообразна в силу отсутствия значимых различий в стиле текстов и

целевой аудитории.

Структурная разметка маркирует главы, абзацы, предложения и словоформы [28]. Для проведения структурной разметки необходимо обосновать процедуру сегментации текста, т. е. его разбиение на необходимые и достаточные сегменты, релевантные для его компьютерной обработки, позволяющие пользователю определить лексические, грамматические и другие свойства изучаемых лингвистических единиц [29].

Терминологическая разметка предназначена для фиксации в тексте имен понятий предметной области [30]. Терминологическая разметка позволяет не только фиксировать присутствие определенных терминов в тексте, но и [31]:

— определять максимальный текстовый фрагмент, представляющий сущность. При этом остается возможность указывать и вложенные фрагменты, представляющие самостоятельные сущности;

— сопоставлять аннотацию самой сущности, а не всей синтаксической группе, в которую она входит;

— использовать при необходимости разрывные фрагменты, в том числе при перечислении с сочинительным сокращением.

Исходя из приведенного анализа видов разметки система управления корпусными данными параллельных корпусов технических текстов должна реализовывать следующие виды разметки: анафорическую, метатекстовую, морфологическую, семантическую, синтаксическую, структурную и терминологическую. При этом как было отмечено выше, существует большое количество автоматических морфологических анализаторов, которые, однако, чаще всего реализуют разметку частей речи [32]. Далее будут рассмотрены существующие методы построения анафорической, морфологической и синтаксической разметок. Данные виды разметки были выбраны, т. к. морфологическая и анафорическая разметки обеспечивают базовое лексико-грамматическое описание словоформ и разрешение референции, что необходимо для корректного построения синтаксической разметки которая, в свою очередь, формализует грамматические отношения между словами и служит основой для операций выравнивания и дальнейшего анализа и использования текстов в корпусе.

2 Методы построения разметок текстов

2.1 Анафорическая разметка

Как было отмечено выше, анафорическая разметка отвечает за установление референтных связей. Глобально выделяют два вида анафоры: именную, называемую также корефернтной или кореферетностью, и местоименную [33]. Пример именной анафоры был приведен выше. В случае местоименной анафоры анафор выражен местоимением.

Для автоматического разрешения анафоры (определения антецедента — выражения, к которому отсылает анафор) в настоящие время выделяют два подхода:

— разрешение на основе правил [34, 35, 36, 37, 38, 39];

— разрешение на основе методов машинного обучения [33, 35, 39, 40, 41, 42, 43].

Как видно из приведенных источников, в современной русскоязычной среде преобладает подход с использованием машинного обучения, тогда как в англоязычных работах изначально рассматривался подход с использованием системы правил.

2.1.1 Вычисление семантической близости

В [38] описывается сравнение различных метрик семантической близости, которая может быть использована как решающий критерий при выборе одного из множества антецедентов данного анафора. Семантическая близость вычислялась на основе открытой части онтологии русского языка RuThes [44] и текстов корпуса RuCor [45]. В работе выделяются три класса метрик, в зависимости от принципа подсчета, на котором они основаны:

1) метрики, основанные на длине пути $path(c_1, c_2)$ между вершинами онтологии c_1, c_2, соответствующими искомым сущностям [46]. В дальнейшем было предложено два метода нормализации:

— с учетом глубины онтологии D [47]:

$$wp(c_1,\ c_2) = -ln\frac{path(c_1,\ c_2)}{2D} \tag{2.1}$$

— с учетом глубины узлов онтологии $depth(c_i)$ [48]:

$$lc(c_1,\ c_2) = \frac{2depth(lca)}{depth(c_1) + depth(c_2)}, \tag{2.2}$$

где lca — наименьший общий предок узлов c_1 и c_2.

2) метрика, основанная на подсчете информационной содержательности, т. е. частота встречаемости понятия, которая рассчитывается как вероятность его появления в корпусе текстов [49]. Метод расчета семантической близости основанный на этой величине

описан в [50] и использует информационную содержательность наименьшего общего предка вершин c_1 и c_2, соответствующих искомым сущностям:

$$res(c_1,\ c_2) = 1 - \frac{ln(hypo(lca) + 1)}{ln(max_{wn})}, \tag{2.3}$$

где lca — наименьший общий предок, $hypo(n)$ — число гипонимов узла, max_{wn} — число вершин в онтологии.

3) метрика, основанная на расчете текстовых пересечений, предложенная в [51]. Ее вычисление основано на количестве пересечений различной длины между текстами:

$$overlap(t_1,\ t_2) = \sum_{i=1}^{n} m^2, \tag{2.4}$$

для m n-словных пересечений между текстами t_1 и t_2. В статье [52] предлагается метод нормализации данной метрики с учетом длины сравниваемых текстов:

$$lesk(t_1,\ t_2) = th(\frac{overlap(t_1,\ t_2)}{length(t_1) + length(t_2)}), \tag{2.5}$$

где $overlap(t_1,\ t_2)$ определяется в соответствии с 2.4, а $length(t_i)$ — длина фрагмента текста $t_i,\ i = \overline{1, 2}$

Приведенные в [38] результаты сравнения указанных метрик, отражены в таблице 2.1. Метрика lesk не могла быть подсчитана на данных из RuThes, поскольку глос сы в нем приведены лишь для очень небольшого числа сущностей [38].

Таблица 2.1 — Значения коэффициента корреляции метрик семантической близости с кореферентной разметкой

Источник	*path*	*wp*	*lc*	*res*	*lesk*
RuThes	0.56	0.51	0.59	0.30	n/a

2.1.2 ABBYY Compreno

Технология ABBYY Compreno описана в [34, 53, 54]. Поскольку данная технология является проприентарной, достоверно неизвестно, какие именно механизмы используются для разрешения кореференции и анафоры.

Однако, исходя из текста статьи, можно сделать вывод, что для разрешения анафоры используется построенное семантическое дерево предложения, в которое дополнительно включаются связи, не принадлежащие исходному дереву. Кроме того, анафора заменяется антецедентом. Исходное семантическое дерево предложения: «Мальчик дал девочке свое яблоко», — представлено на рисунке 2.1.

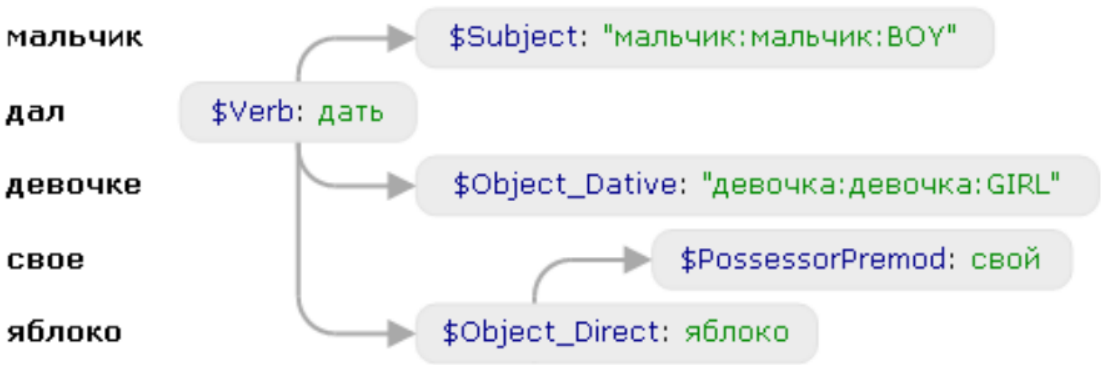

Рисунок 2.1 — Семантическое дерево, построенное системой Compreno

На рисунке 2.2 представлено семантическое дерево с дополнительной связью от антецедента к анафору, при этом анафор заменен на антецедента.

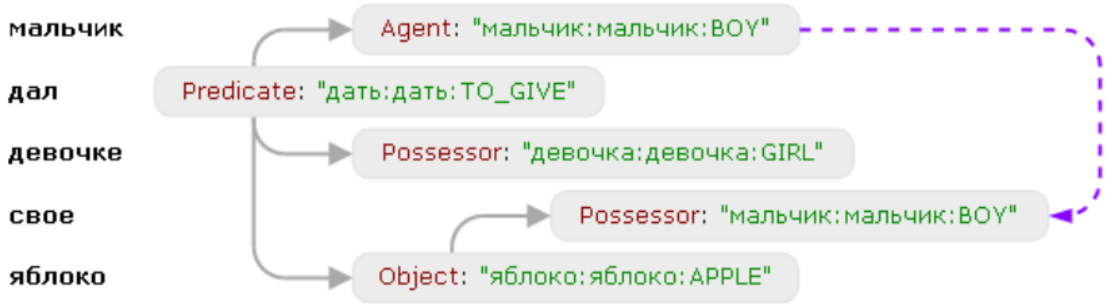

Рисунок 2.2 — Семантическое дерево, построенное системой Compreno, с дополнительными связями и подстановкой антецедента

Для реализации автоматизированного построения семантического дерева требуется корректная морфологическая и синтаксическая разметки, разрешение неоднозначности в которых требует, в свою очередь, наличия семантической разметки, что приводит к замкнутому кругу [1].

В [34] также приведены общие сведения о способе разрешения кореференции между именами собственными и их кореферентами. Тем не менее, сделать какой-либо конкретный вывод о принципах работы данного механизма не представляется возможным.

2.1.3 CogNIAC

CogNIAC является алгоритмом разрешения анафоры в английском языке, построенным на системе правил [37]. Для выполнения задачи алгоритму требуется следующая информация:

— разделенный на предложения текст;
— выделенные именные группы, которые могут быть антецедентами;
— морфологическая разметка, в т. ч. определение части речи, род, число;
— в одной из конфигураций — синтаксические деревья.

Основу алгоритма составляют 6 правил, перечисленные ниже.

1) Уникальность в тексте: если существует единственный возможной антецедент в прочитанной части текста, то данный антецендент выбирается как истинный.

2) Возвратность: истинным выбирается ближайший возможный антецедент в прочитанной части текста, если анафор является возвратным местоимением.

3) Уникальность в текущем и предыдущем предложении: если существует единственный возможный антецедент в предыдущем и прочитанной части текущего предложения, то данный антецендент выбирается как истинный.

4) Притяжательность: если анафор выражен притяжательным местоимением, и существует единственый антецедент в притяжательном падеже в предыдущем предложении, то данный антецедент выбирается как истинный.

5) Уникальность в текущем предложении: если в прочитанной части текущего предложения существует единственный возможный антецедент, то данный антецедент выбирается как истинный.

6) Уникальность подлежащего / местоимения в подлежащем: если подлежащее предыдущего предложения содержит единственный возможный антецедент и анафор является подлежащим текущего предложения, то такой антецедент выбирается истинным.

Метод разрешения анафоры, используемый CogNIAC, работает следующим образом: местоимения обрабатываются слева направо, в порядке их появления в тексте; для каждого местоимения правила обрабатываются в порядке, в котором они приведены выше; для данного правила, если истинный антецедент найден, осуществляется соответствующая разметка и остальные правила не применяются к данному местоимению, иначе проверяется следующее правило; если ни одно ищ правил не подошло, анафора остается неразрешенной.

Данный алгоритм имеет некоторые фундаментальные ограничения. Одним из них является невозможность корректной обработки анафоры в предложениях вида [55]: «The city council refused to give the women a permit because they {feared/advocated} violence». Связано это с тем, что выбор верного антецедента здесь связан с употребляемым глаголом, в зависимости от которого, верным будет либо «The city council», либо «the women», однако, CogNIAC не учитывает значений глаголов в своих правилах. Другим ограничением является предположение о том, что антецедент находится в уже прочитанной части текста в пределах двух предложений от анафора. Однако, как отмечено в [35], антецедент может находиться на удалении до 17 предложений от анафора. Наконец, отмечается [37], что алгоритм не способен обработать множество антецедентов, имеющих общий анафор, например в отрывке: «John called Mary. They went to a movie».

2.1.4 Метод опорных векторов

В [43] описано применение метода опорных векторов для разрешения анафоры местоимений третьего лица. В качестве входных данных были выбраны признаки текущего анафора

и признаки гипотетических антецедентов, согласованных с анафором в роде и числа и встречавшихся ранее в тексте. В качестве признаков были выбраны:

— число имен собственных между анафором и антецедентом;

— количество предложений, разделяющих анафору и антецедент;

— стоит ли антецедент в именительном падеже;

— является ли антецедент именем собственным;

— количество существительных и местоимений, расположенных в предложениях между рассматриваемыми анафором и антцедентом;

— совпадает ли падеж анафора и антецедента;

— статистическая информация о том, в каком сегменте предложения располагается антецедент — насколько близко к началу;

— статистическая информация о том, в каком сегменте предложения располагается анафор — насколько близко к началу;

— количество анафор, реферирующих с текущим антецедентом по данным ручной разметки, расположенных между анафорой и антецедентом;

— число глаголов в сегменте, содержащем антецедент;

— число причастий и деепричастий в сегменте, содержащем антецедент;

— число местоименных прилагательных и союзов в сегменте, содержащем антецедент;

— число существительных в именительном падеже в сегменте, содержащем антецедент;

— род, падеж и число анафоры и антецедента (в виде набора бинарных признаков).

Исходный текст проходил предварительную морфологическую, синтаксическую и первичную семантическую разметку. Анафора местоимений третьего лица с существительными или другими местоимениями, в случае комплексной референции, была размечена вручную.

Каждая пара анафора-гипотетический антецедент может быть отнесена к одному из двух классов, определяемых наличием или отсутствием референции. Утверждается [43], что среди гипотетических антецедентов, предшествующих данному анафору обязательно найдется хотя бы один, связанный с ним референцией. Можно также полагать, что такой антецедент единственен, поскольку в силу транзитивности самого понятия анафоры, связность анафора со всему предшествующими антецедентами гарантируется. Для разрешения анафоры использовался следующий алгоритм:

1) фиксируется очередной анафор и присваивается $n = 1$;

2) осуществляется переход к следующему n-ому гипотетическому антецеденту, согласованном в роде и числе, расположенному ранее в тексте, начиная с ближайшего к рассматриваемому анафору;

3) для пары анафор-гипотетический антецедент запускается метод опорных векторов;

4) выходные данные метода опорных векторов преобразуются в оценку связанности данной пары референтной связью с использованием сигмоиды;

5) n увеличивается на единицу. Если n не превышает числа гипотетических антецедентов, осуществляется переход к шагу 2, иначе — к шагу 6;

6) полученные оценки умножаются на релаксационные коэффициенты r_i, отражающие априорные знания о статистическом распределении анафор;

7) рассматриваемый анафор связывается с m-ым антецедентом, где $m = argmax_n s_n$. Переход к шагу 1.

Поскольку чаще всего [43] анафор реферирует с ближайшим антецедентом, и вероятность референции падает с ростом n, коэффициенты релаксации определялись следующим образом:

$$r_n = \Theta \frac{1}{N} + (1 - \Theta) V_n, \tag{2.6}$$

где коэффициент регуляризации Θ определялся с помощью кросс-валидации, а значение N было принято равным 4.

Точность работы классификатора составила 62%.

2.1.5 Дерево принятия решений

В [41,56] описаны подходы к разрешению анафоры с использованием деревьев принятия решений.

В [41] использовался корпус состоящий из новостных публикаций и художественных текстов, разметка анафоры в которых проводилась вручную. В качестве классификатора по признаку наличия анафоры использовалось дерево принятия решений, обученное по алгоритму ID3 [57].

Признаки, используемые для классификации пары из анафора и возможного антецедента были разделены на три группы:

— линейные:

1) количество имен собственных между анафором и антецедентом;
2) количество предложений между анафором и антецедентом;
3) количество потенциальных анафоров для данного антецедента между ним и данным анафором;
4) количество существительных между анафором и антецедентом;
5) количество анафорических местоимений между анафором и антецедентом;
6) количество потенциальных антецедентов между данным анафором и данным антецедентом;

— морфологические:

1) часть речи антецедента;
2) находится ли антецедент в именительном падеже;
3) количество глаголов в предложении, содержащем антецедент;
4) количество существительных в предложении, содержащем антецедент;
5) количество союзов и местоименных прилагательных в предложении, содержащем антецедент;

6) количество неличных глагольных форм в предложении, содержащем антецедент;

— синтаксические:

1) является ли антецедент подлежащим;

2) является ли анафор подлежащим.

Перед обучением модели, была проведена обработка входных данных, включавшая следующие шаги:

— бинаризация линейных признаков в признаки вида 'более чем X', 'между X и Y' и т. д.;

— удаление всех пар, содержащих анафор, для которого антецедент заведомо не может быть найден;

— все пары в которых антецедент находится на расстоянии, превышающем некоторое пороговое значение, от анафора, отмечаются как отрицательные;

— для всех возможных антецедентов были добавлены признаки, сравнивающие их с ближайшим возможным антецедентом.

Для разрешения анафоры использовалась следующая последовательность действий:

1) автоматически выделяется множество анафоров и множества их возможных антецедентов с помощью Томита-парсера [58];

2) генерируются все возможные пары анафор — возможный антецедент;

3) дерево принятия решений применяется последовательно к каждой паре, начиная с той, в которой содержится ближайший к анафору возможный антецедент. Далее пары рассматриваются по увеличению удаления возможного антецедента от анафора;

4) как только найдена пара классифицированная как анафорическая, поиск останавливается, все последующие пары помечаются как не анафорические, алгоритм переходит к рассмотрению следующего анафора.

На тестовой выборке точность работы данного классификатора составила 52.04%.

Описанный в [56] метод также основывается на деревьях принятия решений. В качестве корпуса использовался вручную размеченный экспертами набор текстов по криминалистике. В качестве классификатора использовался ансамблевый алгоритм Extra Trees [59], а для борьбы с дисбалансом классов в ходе обучения использовался алгоритм Balance Cascade [60].

Всего использовалось 240 признаков, объединенных в следующие группы:

— бинарные категориальные признаки: согласование в роде, числе, падеже, является ли антецедент одушевленным лицом. Данные признаки были получены с использованием MyStem [61], всего 7 признаков;

— небинарные категориальные признаки: часть речи, падеж, род, число. Данные признаки были получены с помощью MyStem и SyntaxNet [62], всего 82 признака;

— числовые признаки: все возможные расстояния между контекстными векторами анафора и возможных антецедентов, расстояние между собственными векторами анафора и возможных антецедентов. Данные признаки выделены с помощью Word2Vec [63],

SyntaxNet, всего 28 признаков;

— числовые линейные признаки: расстояния в словах, предложениях, существительных, глаголах и т. д. между возможным антецедентом и анафором. Данные признаки получены с помощью MyStem, всего 13 признаков;

— TF-IDF векторы для анафора и возможного антецедента. Данные признаки получены с помощью SyntaxNet, всего 110 признаков.

Алгоритм разрешения анафоры состоит в следующем:

— для каждого личного местоимения в пределах двух предложений формируются все возможные пары с антецедентами;

— для каждой пары генерировался вектор из описанных выше признаков;

— обученный ансамбль оценивал для каждого антецедента вероятность того, что данная пара является истинной;

— для данного анафора выбирается антецедент с наибольшей вероятностью, среди всех антецедентов, чья вероятность превышает некоторое пороговое значение.

На тестовой выборке описанная модель, в зависимости от порогового значения вероятности, показала результаты, представленные в таблице 2.2.

Таблица 2.2 — Результаты модели в зависимости от порогового значения вероятности

Пороговая вероятность	Precision	Recall
0.280	0.6577	0.7789
0.285	0.6605	0.7748

2.2 Морфологическая разметка

В настоящее время, морфологическая разметка выполняется автоматически в большинстве случаев [32], однако требует верификации экспертом лингвистом, так как при наличии, например, омонимии вероятность ошибки автоматического разметчика повышается [64].

Выделяют следующие подходы к автоматическому выполнению морфологической разметки:

— на основе конечных автоматов (FST) [65, 66, 67];

— словарный подход с использованием эвристик [68, 69, 70, 71];

— машинное обучение [72].

2.2.1 Finite state transducer (FST)

В [66] описан морфологический процессор на основе конечных автоматов с выходом для языка хинди. Данный процессор состоит из двух компонентов: анализатора и генератора. Оба в свою очередь требуют лексикона, т. е. словаря корней слов, файлов описывающих переходы

конечных автоматов и словаря неизменяемых слов. Лексикон был составлен вручную, всего включал в себя 91390 слов. Файлы содержащие флективные и деривационные правила словообразования, т. е. правила переходов конечных автоматов были созданы с помощью инструментов Stuttgart Finite State Transducer (SFST) [73]. Задачей анализатора является получение леммы и морфологических характеристик слова по его словоформе. Задача генератора противоположна: получение словоформы по заданным морфологическим признакам и лемме.

В статье [66] приведено значение 97% корректных результатов на выборке размером 4000 словоформ.

В [65] приведено описание морфологического анализатора для языка мапуче — изолированного языка индейцев мапуче Южной Америки. Основой для анализатора послужила грамматика «A Grammar of Mapuche» Инеке Смитс [74]. Анализатор состоит из следующих частей:

— непосредственно finite state transducer — скомпилированный с помощью инструментов FOMA [75] конечный автомат, преобразующий словоформу в лемму и набор морфологических признаков;

— словарь корней разделенный по частям речи;

— словари аффиксов сгруппированные по позициям в слове и придаваемому значению;

— файлы, описывающие регулярные выражения, задающие допустимые комбинации морфем;

— файлы, описывающие регулярные выражения, задающие изменения в написании слов на стыке морфем;

— препроцессор-унификатор письменности мапуче.

Тестирование анализатора производилось на примерах из грамматики Смитс. Автор не приводит полученных значений точности, однако, приводятся примеры словоформ не распознанных системой. В частности были выделены:

— пропущенные в словаре или альтернативные формы в лексиконе;

— расхождения в описании морфем в грамматике Смитс и других авторов;

— словоформы, составленные по более сложным правилам, не описанным в анализаторе.

В [67] описан морфологический анализатор для центрально-аляскинского юпика. В основе анализатора — грамматические словари языка.

Анализатор состоит из 6 последовательно используемых уровней:

— первый уровень анализатора убирает возможный дефис энкликтики т. е. слово входящее в одно фонетическое слово с предыдущей словоформой;

— второй уровень составляет слово, разбитое на морфемы. Данный уровень ищет некоторое представление слова в виде морфем, которое посредством применения просодических правил может быть преобразовано к исходному слову;

— третий уровень снимает диакритики ударения, оставляя морфемное представление без них;

— четвертый уровень составляет морфемное представление с учетом подходящих мор-

фонологических правил. То есть из входного представления слова, разделенного на морфемы, получается деление на морфемы с добавлением специальной разметки, указывающей возможное применение морфологических правил на стыках морфем;

— пятый уровень заменяет конкретные алломорфы на обобщенные. Таким образом, на выходе данного уровня получается цепочка морфем в том виде, в котором они встречаются в словаре, соединенных разметкой указывающей правила, применяемые на стыках морфем;

— шестой уровень удаляет разметку границ морфем, а также осуществляет поиск морфем в словаре. Итогом этого этапа является морфологическая разметка слова.

Исходный текстовый корпус, на котором производилась проверка анализатора был составлен на основе записей устной речи носителей языка, а также перевода Библии на юпикский язык. При этом разметка была выполнена вручную автором для 300 случайных слов, на которых проверялась точность работы анализатора. В 70% случаев среди выданных анализатором разборов присутствовал правильный. Дополнительно, автор оценил покрытие, т. е. процент случаев, когда анализатор составлял хотя бы один разбор слова. Покрытие составило 98.1% для устных текстов и 90.4% для перевода Библии.

2.2.2 Словарные подходы с использованием эвристик

Морфологический анализатор mystem был разработан И. В. Сегаловичем для поисковой системы Яндекс [68]. Основная идея алгоритма в использовании грамматического словаря для поиска в нем нужных словоформ или сопоставления неизвестной словоформы с уже существующими, если нужная словоформа в словаре не найдена. Такой подход обоснован следующими проблемами, встречающимися при обработке поисковых запросов:

— большое количество неизвестных слов встречающихся в поисковых запросах, например, опечатки, неологизмы и иностранные слова;

— отсутствие универсальной морфологической модели для разных языков;

— необходимость высокой скорости обработки, без использования контекста.

Алгоритм использует наборы префиксных деревьев для всех инвертированных корней и суффиксов. Исходные словоформы были взяты из грамматического словаря А. А. Зализняка [76]. Алгоритм работает следующим образом:

1) первым шагом алгоритма является поиск всех возможных суффиксов в префиксном дереве суффиксов. Поиск осуществляется с конца слова, таким образом получаются все возможные границы между корнем и суффиксом в слове;

2) для каждой возможной границы, начиная с самого длинного суффикса производится поиск подходящей основы.

3) если терминальный лист в префиксном дереве корней найден, проверяется есть ли словоформа с таким корнем и суффиксом в словаре;

4) если для всех возможный границ не найдено подходящего корня или словоформа не встречается в словаре, то алгоритм начинает проверять все возможные корни начиная от самой глубокой точки ветвления в дереве корней. Таким образом наиболее подходящим корнем будет считаться тот, у которого подотрезок совпадающих с исходной словоформой символов на конце максимален.

Для оценки точности алгоритма использовалась метрика семантической близости пар потенциальных морфологический вариантов (PPMV). Такой парой считаются две словоформы обладающие одной леммой или основой слова. Для каждой словоформы вычислялось количество добавленных пар, т. е. таких словоформ, которые анализатор посчитал обладающими одной леммой при различных леммах, и потерянных пар, т. е. словоформ, которые анализатор посчитал обладающими различными леммами при общей лемме. Эталонными считались пары обладающие общей леммой в НКРЯ. Для добавленных и потерянных по сравнению с эталонами из НКРЯ словоформ производилось сравнение с результатами поисковых запросов в Google. Для добавленных пар:

1) для каждого слова пары выполнялись поисковые запросы в Google и вычислялось число общих результатов поиска;

2) если число общих результатов поиска было мало, такая пара признавалась добавленной, иначе нет.

Для потерянных пар поисковые запросы выполнялись для словоформы и ее эталонной пары. Если в результате количество совпадающих результатов было велико, такая пара признавалась потерянной. В результате были получены значения в 778 добавленных пар и 41 потерянных.

Также была проведена оценка быстродействия алгоритма. Замерялась скорость обработки слов на выборке из 1 миллиона словоформ. Алгоритм показал скорость обработки в 60 – 65 тысяч словоформ в секунду.

Алгоритм стемминга Портера был опубликован в 1980 году [77], последняя правка в алгоритм была внесена автором в 2000 году [70]. Данный алгоритм основан на правилах, составленных из статистических наблюдений, при этом в контексте данного алгоритма под суффиксом подразумевается подотрезок букв в конце слова, а не суффикс в лингвистическом смысле. В дальнейшем данный алгоритм был переработан для использования в других языках, например, в русском [78].

Основная идея алгоритма — последовательное отсечение суффиксов слова, до момента получения корня слова. Последовательность удаления такова:

1) удаление суффиксов слова, соответствующих совершенному причастию. Если данные суффиксы не найдены, удаляется возвратный суффикс и затем суффикс прилагательного, глагола или существительного;

2) если полученное слово оканчивается на «и», она удаляется;

3) если присутствует «словообразовательный» суффикс (d-suffix) после второго сочетания гласный – согласный, необходимо удалить его. К таким суффиксам отнесены сочетания «ост», «ость»;

4) одиночная «н» или если есть окончание превосходной степени («ейш», «ейше») и последующая одиночная «н» удаляются. В противном случае, если слово оканчивалось на «ь» он также удаляется.

В статье [68] для сравнения также были произведены замеры реализации алгоритма стемминга Портера. Было получено 643 добавленных пары потенциальных морфологических вариантов и 487 потерянных пар. Оценка быстродействия составила 180 – 190 тысяч слов в секунду.

Как отмечено в [69] основным недостатком данного алгоритма является то, что он всегда выдает один вариант корня слова, а следовательно, один вариант морфологического разбора.

Морфологический анализатор stemka был описан А. Коваленко в 2002 году [69]. Основная идея данного алгоритма состоит в выборе наиболее вероятного разделения на основу и суффикс слова, называемый в статье окончанием, т. е. непрерывную последовательность символов в конце слова. Для вычисления вероятностей алгоритм разбирал базу из 4 миллионов словоформ, содержащих 150 тысяч основ. Разбор производился по следующим принципам:

— выделялся некоторый суффикс и две предшествующие суффиксу буквы основы. Если такое сочетание суффикса и окончания основы уже известно анализатору, его вес увеличивается. В противном случае новое сочетание заносится в таблицу таких инвертированных сочетаний;

— дополнительным требованием к разбиению на пары было наличие хотя бы одной гласной в основе;

— полученный набор пар суффикс и окончание основы был отсортирован и все пары, вероятность реализации которых составила менее 0,001 были исключены.

Таким образом каждое новое анализируемое слово также последовательно разбивается на пары суффикс и двухбуквенное окончание основы. Для каждой пары производится ее поиск в уже имеющейся таблице. Если такое вхождение содержится в таблице, то данный вариант разбора помечается как подходящий, иначе производится следующее разбиение. Благодаря такому принципу поиска алгоритм может определить несколько возможных вариантов морфологических характеристик слова.

В [68] были произведены замеры для данного алгоритма. При этом отдельно производились замеры для выбора наиболее длинного (aggressive) и наиболее короткого (conservative) суффикса в выдаче алгоритма. Результаты замеров для обоих этих случаев приведены в таблице 2.3.

Таблица 2.3 — Показатели семантической близости пар потенциальных морфологических вариантов для двух типов выдачи алгоритма

Тип выдачи	Добавленные пары	Потерянные пары
Aggressive	787	493
Conservative	136	1384

Результаты замеров быстродействия алгоритма составили 380-390 тысяч слов в секунду.

В [71] приведено описание внутреннего устройства морфологического анализатор pymorphy2. Данный анализатор основан на грамматическом словаре русского языка Зализняка [76]. Внутреннее представление морфологии слова в данном анализаторе основано на парадигме — наборе префиксов и суффиксов (в смысле непрерывных последовательностей букв в начале и конце слова соответственно) всех словоформ данного слова, сопровождаемых тэгами, отражающими морфологические характеристики данного слова. При этом одна и та же парадигма может соответствовать различным словам, таким образом для порядка 400 тысяч лексем получается около 3 тысяч парадигм. При этом слова представлены в виде детерминированного ациклического конечного автомата (DAFSA), терминальная вершина в котором, содержит ссылку на парадигму, соответствующую данному слову, а также указание номера конкретной словоформы в данной парадигме.

Таким образом, поиск слов известных словарю осуществляется в DAFSA. В случае слов неизвестных словарю анализатор пытается с помощью эвристических правил определить парадигму, соответствующую данному слову.

Первым применяется отсечение известных словарных префиксов. Предполагается, что отсечение или добавление такого префикса не изменяет морфологических характеристик слова. При успешном удалении, производится повторный поиск в DAFSA. В противном случае используется отсечение неизвестных префиксов. Данное правило основывается на предположении, что два слова, отличающиеся только некоторым префиксом, имеют одинаковые морфологические характеристики. Таким образом, если в словаре найдено слово, соответствующее некоторому суффиксу анализируемого, то используются используется его разбор. При этом накладываются следующие ограничения:

— словарное слово должно состоять не менее, чем из 3 букв;

— неизвестный префикс должен состоять не более, чем из 5 букв;

— словарное слово должно быть одной из следующих частей речи: существительное, прилагательное, глагол, причастие или деепричастие.

Длины проверяемых неизвестных префиксов составляют 1, 2, 3 или 5 букв.

Тем не менее предположения, на которых основывается отсечение префиксов не всегда соответствуют действительности. Например, приставка «с-» у глаголов изменяет вид. Для таких словоизменительных префиксов строится отдельный DAFSA, в случае успешного поиска по которому, соответствующие морфологические характеристики добавляются к вариантам разбора.

Когда обработка префикса не дает результата, необходимо научиться определять морфологические характеристики несловарного слова по его суффиксу. Для этой цели при первом чтении словаря для каждого суффикса длиной от 1 до 5 букв ставится в соответствие частота, ссылка на парадигму и номер словоформы в парадигме. Полученные таким образом сочетания суффикса и парадигмы фильтруются по следующим правилам:

— сохраняется только самое частотное соответствие для каждой части речи;

— соответствия, содержащие редко встречающиеся парадигмы, отсеиваются;

— соответствия, содержащие редко встречающиеся суффиксы, отсеиваются;

— соответствия, которые приводят к добавлению суффиксов предлогам, частицам, междометиям и другим подобным частям речи также отсеиваются.

Для суффиксов оставшихся после фильтрации парадигм, строится DAFSA, в котором при обработке несловарного слова ищется суффикс максимальной длины.

Отдельно обрабатываются наречия. Несловарное слово определяется как наречие, если выполняются условия:

— слово начинается на «по-»;

— слово длиннее 5 букв;

— часть слова без «по-» может быть разобрана как прилагательное единственного числа дательного падежа.

Версия pymorphy 0.5.6 при полностью загруженном в память словаре обеспечивает скорость обработки в 2 тысячи слов в секунду.

2.2.3 Статистические подходы

В [72] описана модель, использующая метод опорных векторов (SVM), для морфологической разметки. Основой для данной модели послужил инструмент морфологической разметки SVMTool [79], который тем не менее обладал рядом существенных недостатков для русского языка:

— отсутствие возможности обрабатывать «сложные» тэги, например, вложенные;

— использование библиотеки SVMLight, как единственной доступной в Perl, на котором реализован SVMTool, которая не всегда оптимальна для поставленной задачи по мнению авторов;

— отсутствие возможности интеграции с другой существующей системой морфологического анализа STAGE.

Для обработки слова и получения его морфологических признаков моделью, используется контекстное окно (в дальнейшем называемое просто контекст) охватывающее 7 слов, по три слева и справа от анализируемого.

В качестве признаков для классификации слова были выбраны:

— словоформы всех слов в контексте, а также их биграммы и триграммы;

— одно-, двух-, трех- и четырехбуквенные префиксы и суффиксы всех словоформ в контексте. Этот признак отмечен как наиболее полезный для неизвестных словоформ;

— тэги уже размеченных словоформ, их биграмм и триграмм в контексте;

— вложенные тэги уже размеченных словоформ в контексте;

— возможные тэги для всех словоформ в контексте;

— возможные вложенные тэги для всех словоформ в контексте;

— классы неоднозначности для всех словоформ в контексте, их биграмм и триграмм.

Классы неоднозначности представляют собой конкатенацию всех возможных тэгов словоформы, при этом возможные тэги доступны из словаря еще до обработки слова моделью. Таким образом модель может опираться не только на уже размеченные данные слева от анализируемого слова, но и на возможные варианты разметки справа от него.

Для реализации итогового классификатора было выбрано два подхода: классификатор полных тэгов и классификатор вложенных тэгов. Классификатор полных тэгов предполагает обучение отдельного классификатора для каждого возможного тэга. В результате каждый из около 500 классификаторов возвращает определенное значение (decision value, dv) отражающее, насколько данный тэг применим к анализируемому слову. Выбирается тэг, соответствующий классификатору, вернувшему максимальное значение dv. Именно такой подход реализован в исходном инструменте SVMTool. Классификатор вложенных тэгов предполагает обучение классификатора для каждого варианта вложенного тэга. Таким образом число необходимых классификаторов сокращается до порядка 50 штук. В результате итоговый тэг должен быть сформирован на основе вложенных тэгов. Авторами приведено два варианта его получения. Первый заключается в последовательном отсечении для каждого вложенного тэга всех полных тэгов, в которые он входить не может. Второй заключается в формировании полного тэга как составленного из вложенных тэгов с максимальным dv, исходя из предположения, что вложенные тэги независимы. Хотя предположение не является верным, данный вариант работает не хуже первого, кроме того быстрее него [72].

Обучающая и тестовая выборки формировались на основе морфологически размеченной части корпуса СинТагРус [80]. Обучающая выборка состояла из 720 тысяч слов, обучающая — 75 тысяч слов. Для проведения экспериментов авторы составили отдельные обучающую и тестовые выборки размерами 440 и 56 тысяч слов соответственно. В результате были сформулированы следующие выводы:

— неверная разметка неизвестных слов, т. е. таких, которые не встречались в обучающей выборке и для которых отсутствует набор возможных тэгов, не вносит существенного влияния на разметку известных слов. В случае, когда все неизвестные слова искусственно размечались правильно, accuracy для известных слов составила 92.47%. В противном случае, когда неизвестные слова размечались самой моделью, и accuracy для них составила 7.25%, эта же метрика для известных слов понизилась до 92.38%;

— классификатор вложенных тэгов показал наибольшую точность (метрика accuracy составила 92.47%);

— классификатор полных тэгов показал большую точность для неизвестных слов, accuracy составила 73.02% против 69.18% у классификатора вложенных тэгов.

В [81] приведено описание модели Meta-BiLSTM, являющейся развитием модели описанной в [82], одержавшей победу в соревновании CoNLL 2017. Исходная модель позволяет выполнять разметку частей речи слов в тексте, а также строить граф зависимостей слов в предложении. Каждое слово для данной модели является суммой трех векторных представлений:

— предобученное представление из внешней модели;

— обучаемое представление, встречающееся в обучающей выборке выше заранее определенного порога;

— символьное представление, которое строится на основе всех символов слова, с помощью односторонней LSTM, отображено на рисунке 2.3.

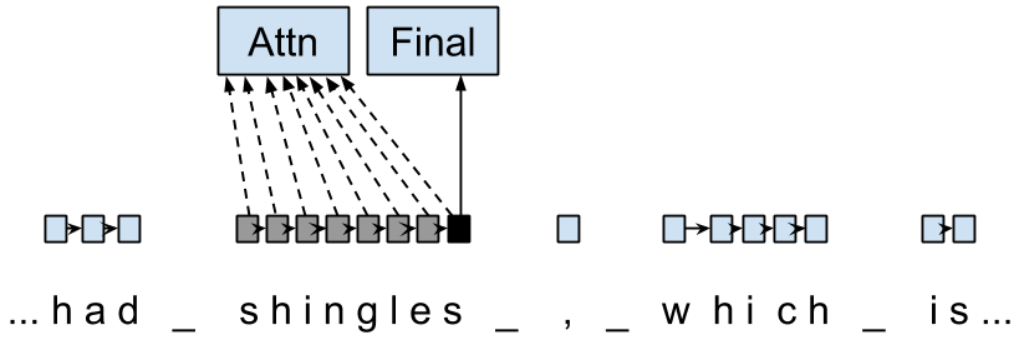

Рисунок 2.3 — Векторное представление, подаваемое на вход LSTM формируется как конкатенация вектора, полученного с использованием механизма внимания для всех символов слова кроме последнего, и обучаемого вектора последнего символа слова

Полученное таким образом векторное представление слова поступает на вход двухслойной BiLSTM. Выходом данной модели является вектор скрытых состояний, представляющий собой конкатенацию векторов скрытых состояний прямого и обратного прохода BiLSTM. Данный вектор поступает на вход полносвязному слою (MLP), после чего выход этого слоя анализируется линейным классификатором, который строит распределение вероятностей по всем возможным тэгам частей речи. В качестве результата выбирается тэг с максимальной вероятностью. Архитектура данной модели представлена на рисунке 2.4.

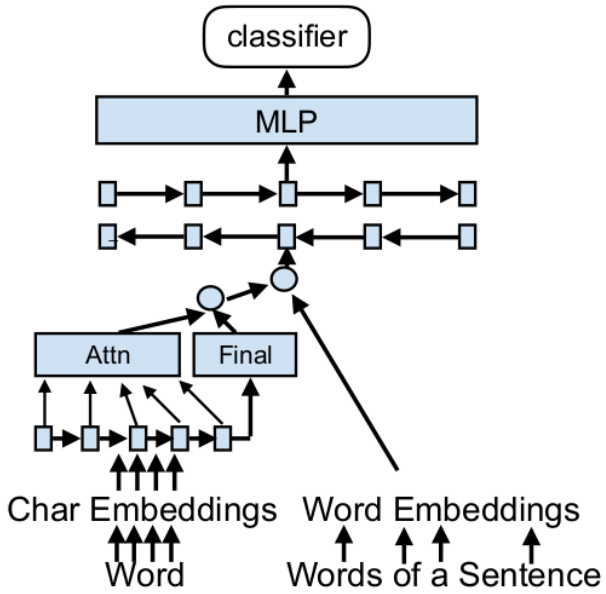

Рисунок 2.4 — Архитектура модели описанной в [82]

Модель, описанная в [81] использует другой подход к обработке символов слова, позволяет при этом получить тэги, содержащие больше морфологической информации, чем просто

часть речи. Векторное представление слова формируется в ней также из трех отдельных представлений.

Символьное представление формируется на основе векторных представлений каждого символа в предложении. Данные представления символов поступают на вход двухслойной BiLSTM. Для формирования итогового представления используется конкатенация прямого и обратного представлений первого и последнего символа слова. Полученный вектор подается на вход MLP слою, как это показано на рисунке 2.5. К выходу данного слоя применяется функция активации softmax, которая позволяет получить распределение вероятности для всех тэгов.

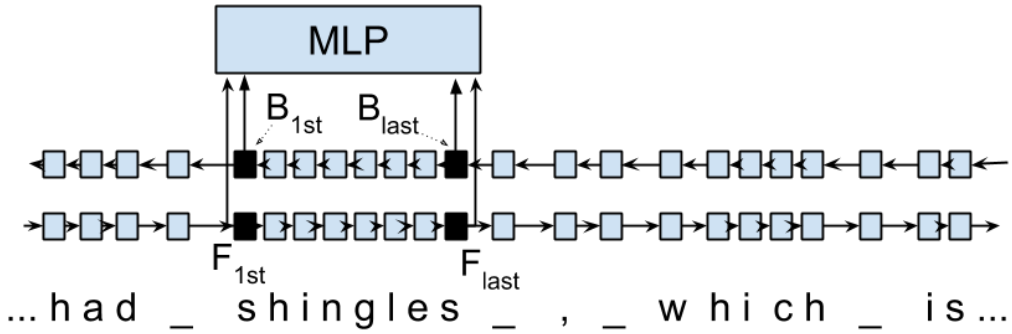

Рисунок 2.5 — Формирование символьного представления на основе всех символов предложения

Предобученное и обучаемое векторные представления суммируются и сумма поступает на вход BiLSTM слою. Полученные данным слоем прямые и обратные представления конкатенируются и передаются на вход MLP слою, после чего выход также передается классификатору для определения тэга.

Выходы MLP слоев символьного представления и представлений слова конкатенируются и передаются на вход еще одной BiLSTM, для получения тэгов учитывающих не только контекст слова, но и контекст предложения. Далее, полученные от этого слоя прямые и обратные представления также конкатенируются и передаются MLP слою, по выходу которого классификатор сможет определить результирующий тэг. Полная архитектура модели представлена на рисунке 2.6.

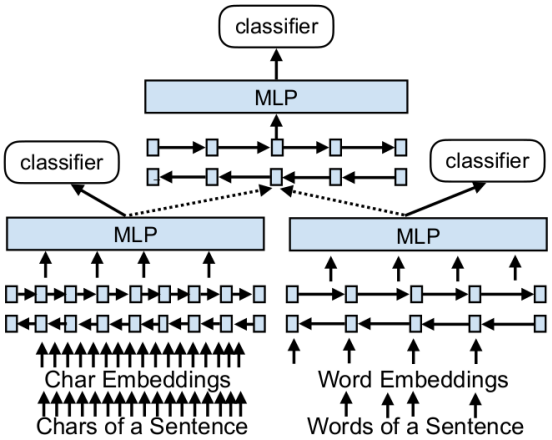

Рисунок 2.6 — Архитектура модели описанной в [81]

Для обучающей и тестовой выборок обеих моделей использовались наборы размеченных текстов на 45 языках, содержащие от нескольких тысяч (старославянский) до миллиарда (английский) словоупотреблений, CoNLL 2017 Shared Task [83]. Для каждой словоформы разметка включает в себя следующие данные:

— лемма;

— универсальная часть речи, т. е. встречающаяся во всех представленных языках;

— специфичная для данного языка часть речи;

— набор морфологических характеристик.

При обучении Meta-BiLSTM модели величины ошибок классификаторов символьного представления и представления слова уменьшаются независимо, при этом наилучшими считаются параметры модели, соответствующие минимальной ошибке классификатора последнего BiLSTM слоя.

Авторы Meta-BiLSTM приводят значения точности (accuracy) для 54 наборов размеченных текстов в определении части речи специфичной для языка и морфологических характеристик слов. В определении части речи модель показала наименьшую точность для вьетнамского — 73.12%, наибольшую — для индонезийского языка — 99.99%, среднее значение составило 93.4%. Модель превзошла победителей CoNLL 2017 в 32 наборах. Наименьшее значение accuracy в определении морфологических характеристик модель показала для иврита — 79.76%, наибольшее — для бразильского португальского — 99.75%, среднее значение — 93.31%. По этому показателю модель превзошла победителей CoNLL 2017 в 43 наборах.

2.3 Синтаксическая разметка

Синтаксическая разметка является первым уровнем разметки работающим не с отдельными словами, а с полноценными предложениями [1]. Данный вид разметки позволяет решать задачи генерации текстов, автоматического реферирования, установления авторства текстов [84, 85], а также исследовать синтаксические конструкции языков [86].

В настоящий момент сложились два подхода к автоматическому построению синтаксической разметки [85]:

— построение дерева составляющих, в котором каждый лист является словом, поддеревья отражают фразовые конструкции в предложении, а дуги между ними — отношения вложенности таких конструкций [87, 88, 89];

— построение дерева зависимостей, в котором вершины соответствуют отдельным словам предложения, а дуги между ними — синтаксическим подчинительным связям между словами [89, 90, 91, 92, 93].

При этом дерево составляющих больше подходит для языков с фиксированным порядком слов, тогда как дерево завсисмостей — для языков со свободным порядком. При этом основной проблемой обоих подходов является обработка однородных членов предложения [85].

2.3.1 Дерево составляющих

В [89] описан алгоритм Кока—Янгера—Касами—Шварца—Зиппеля (CKY) построения дерева составляющих для любой контекстно-свободной грамматики. Основой для алгоритма служит теорема, утверждающая, что любая контекстно-свободная грамматика может быть приведена к нормальной форме Хомского [87].

Алгоритм использует подход динамического программирования. Для предложения состоящего из n словоформ, производится заполнение таблицы dp размером $(n+1) \times (n+1)$ над главной диагональю. Исходные словоформы располагаются на диагонали, $dp_{i,\ j},\ i = \overline{0,\ n},\ j = \overline{0,\ n},\ i < j$ характеризует группу словоформ с i по j исходного предложения. Поскольку в дереве составляющих каждая не листовая вершина имеет двух потомков, $dp_{i,\ j} = dp_{i,\ j} \cup (dp_{i,\ k} \cup dp_{k,\ j}),\ i < k < j$. Тип объединения определяется на основе грамматики. Пример заполнения таблицы dp представлен на рисунке 2.7.

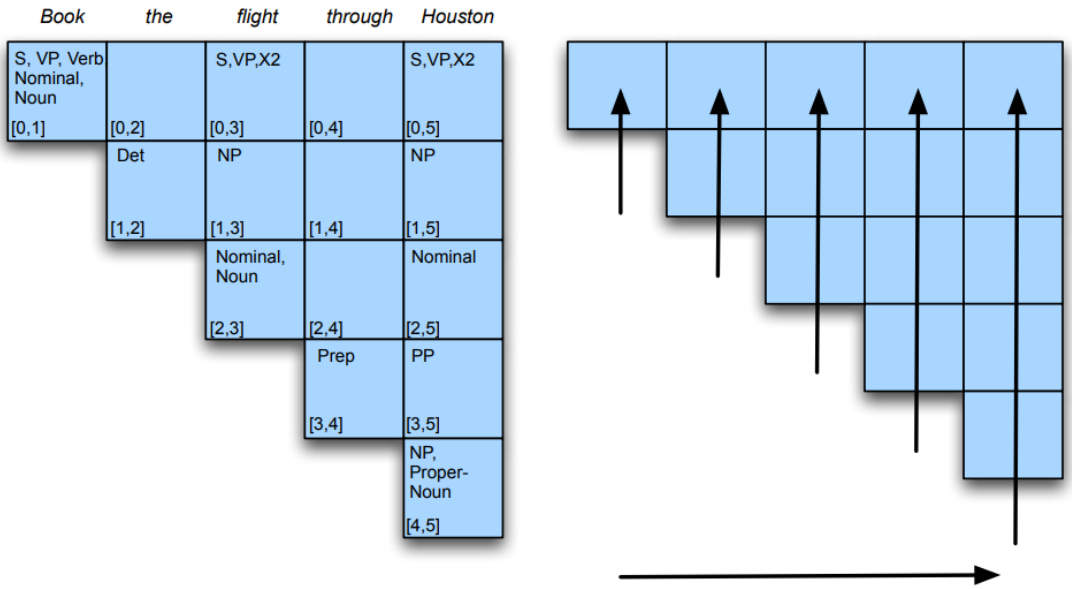

Рисунок 2.7 — Пример заполнения таблицы для предложения «Book the flight through Houston»

Заполнение таблицы dp производится слева направо, снизу-вверх, к моменту вычисления $dp_{i,\ j}$ известны значения всех $dp_{l,\ j},\ l < i$ и $dp_{i,\ m},\ m < j$, соответственно могут быть проверены все возможные разбиения интервала $[i, j]$. Варианты разбора входного предложения, таким образом, находятся в $dp_{0,\ n}$. Выбор верного варианта разбора среди представленных в $dp_{0,\ n}$ осуществляется вручную или предобученным классификатором.

Для построения дерева составляющих по полученной таблице dp необходимо определить алгоритм восстановления ответа. Авторы алгоритма предлагают сохранять для каждого значения $dp_{i,\ j},\ i < j$ указатели на $dp_{i,\ k}$ и $dp_{k,\ j}, i < k < j$, на основе которых оно было было получено.

Вычислительная сложность алгоритма составляет $O(n^3)$, доказательство приводится

в [87].

2.3.2 Дерево зависимостей

В статье [90] описан алгоритм построения дерева зависимостей на примере шведского языка, которому присущ свободный порядок слов. Основой для данного алгоритма послужили shift-reduce алгоритмы, нашедшие применение в компиляторах языков программирования [94].

Алгоритм работает с тройкой:

$$< S,\ I,\ A > \tag{2.7}$$

где S — стек уже обработанных слов, представленный в виде списка,
I — список еще не обработанных слов,
A — множество ребер в графе зависимостей.

Исходное состояние тройки $< nil, W, \emptyset >$, где W — список слов исходного предложения. Алгоритм заканчивает свою работу, когда тройка достигает состояние $< S,\ nil,\ A >$ при любых состояниях стека S и множества дуг A.

Для определения наличия дуги между словами, используется грамматика R, содержащая правила следующего вида:

$$\begin{gathered} w \rightarrow w' \\ w \leftarrow w' \end{gathered} \tag{2.8}$$

если словоформа w предшествует во входном предложении w', где $w,\ w'$ — словарные словоформы. В статье правила описаны оперирующими конкретными словоформами, однако в дальнейшем рассматривается их обобщение на части речи.

Алгоритм определяет 4 правила переходов, представленные в таблице 2.4.

Таблица 2.4 — Правила переходов алгоритма

Shift	$< S,\ w\|I,\ A > \rightarrow < w\|S,\ I,\ A >$	Перенос следующего не обработанного слова из списка I на вершину стека S.
Reduce	$< w\|S,\ I,\ A > \rightarrow < S,\ I,\ A >$	Удаление вершины стека S.
Right-Arc	$< w\|S,\ w'\|I,\ A > \rightarrow < w'\|w\|S,\ I,\ A \cup (w,\ w') >$	Добавление дуги от словоформы w на вершине стека S к первой не обработанной словоформе w'. Словоформа w' переносится на вершину стека S.
Left-Arc	$< w\|S,\ w'\|I,\ A > \rightarrow < S,\ w'\|I,\ A \cup (w,\ w') >$	Добавление дуги от первой не обработанной словоформы w' к словоформе w на вершине стека S. Вершина стека S удаляется.

Исходная версия алгоритма использует фиксированный порядок проверки применимо-

сти правил: Left-Arc → Right-Arc → Reduce → Shift. Однако в дальнейшем рассматриваются варианты алгоритма с дополнительными условиями для разрешения конфликтов между Reduce и Shift, а также между Shift и Right-Arc.

Для оценки точности использовалась выборка из 257 предложений из Stockholm-Umeå Corpus [95], а также грамматика из 126 правил. Точность алгоритма вычислялась как доля слов, для которых был правильно определен предок в дереве зависимостей. Точность исходного алгоритма составила 80% при среднеквадратичном отклонении 13.2%. Точность версии алгоритма с дополнительными условиями выбора правила перехода составила 89% при среднеквадратичном отклонении 10.6%. В статье также приводится доказательство линейной вычислительной сложности алгоритма.

Дальнейшее развитие алгоритм получил в виде отдельного парсера MaltParser [91]. Данный парсер может использовать один из двух алгоритмов построения дерева зависимостей: описанный выше алгоритм, предложенный Ниврe, или алгоритм, предложенный Ковингтоном [92] для обработки непроективных конструкций языка. При этом парсер присваивает дугам в дереве зависимостей метки, характеризующие тип зависимости. Для определения типа зависимости также предлагается два возможных метода: метод k ближайших соседей и memory-based learning или SVM. Отмечается, что оба метода определения типа зависимости дают сопоставимый уровень точности, но конкретные значения не приводятся.

В статье [93] описан паресер LPaRus для русского языка, позволяющий построить дерево зависимостей предложения с метками определяющими тип зависимости. Основным ограничением данного парсера является требование проективности предложения.

Алгоритм работы парсера состоит в постепенном отсечении «невозможных» зависимостей в дереве. Основным понятием является понятие веса дуги, который складывается из:

— некоторой базовой величины, основанной на частоте употребления данного вида зависимости в русском языке;

— длины связи в токенах (зачастую токен состоит из одной словоформы);

— частотность данного сочетания слов, вычисленная на основе больших русскоязычных слов.

На начальном этапе парсер определяет все возможные подлежащие. После чего для каждого подлежащего выбираются все возможные сказуемые, среди которых отсекаются грамматически не согласованные варианты. Среди полученных пар выбирается дуга с минимальным весом.

На следующем этапе парсер определяет для каждой словоформы диапазон, в котором могут находиться ее зависимые. Диапазон определяется на основе требования проективности, т. е. дуги между различными парами главного и зависимого не могут пересекаться, будучи проведенными между словоформами расположенными в исходном порядке, в котором они встречаются в предложении. Среди полученных дуг сразу те, что связывают грамматически несогласованные словоформы. Таким образом парсер строит список пар всех возможных зависимостей.

Далее парсер производит сортировку полученного списка в порядке возрастания весов

дуг. Обход массива начинается от дуг с минимальным весом. Для каждой дуги проверяется возможность включения ее в итоговый разбор. Если дугу возможно включить в разбор, то из списка исключаются варианты противоречащие наличию данной дуги в разборе, начиная с вариантов с наибольшим весом. Противоречие определяется на основании лексико-грамматических критериев, например: словоформа не может быть зависимой от нескольких других; включение данной дуги уточняет грамматических характеристики входящих в нее словоформ, что делает другие варианты грамматически не согласованными; наконец, большая часть вариантов отсекается в силу требований проективности. Таким образом получается набор возможных деревьев зависимостей.

Среди всех полученных деревьев отсекаются не удовлетворяющие одному или более из следующих критериев:

— для каждой словоформы предложения должна быть соответствующая ей вершина в дереве;

— все вершины должны быть достижимы из вершин соответствующих подлежащему и/или сказуемому;

— метки дуг от вершин соответствующих служебным к вершинам, соответствующим самостоятельным частям речи должны соответствовать функциям выполняемым служебной частью речи.

Среди оставшихся вариантов разбора выбирается вариант с наименьшим суммарным весом дуг.

Оценка точности работы парсера производилась на специально собранном корпусе новостных статей агентств «Интерфакс» и «ИТАР-ТАСС» из 800 предложений, средней длинной 20 словоформ. Значение метрики precision составило 81.66%, метрики recall — 85.73%. Основными проблемами для парсера являются сложносочиненные и сложноподчиненные предложения, состоящие из большого числа словоформ, обработка имен собственных и иноязычных вставок.

3 Сравнение методов построения разметки

3.1 Анафорическая разметка

3.1.1 Критерии и сравнение методов

Для сравнения алгоритмов анафорической разметки были выбраны следующие критерии:

— необходимость использования морфологических характеристик;

— необходимость использования синтаксических характеристик;

— возможность применения к различным языкам;

— разрешение местоименной анафоры;

— разрешение кореферентной анафоры;

— точность.

Сравнение методов построения анафорической разметки по приведенным критериям представлено в таблице 3.1

Таблица 3.1 — Сравнение методов построения анафорической разметки

Метод	Требуются морфологические характеристики	Требуются синтаксические характеристики	Разрешение местоименной анафоры	Разрешение кореферентной анафоры	Применимость к различным языкам	Точность
Вычисление семантической близости	Нет	Нет	Нет	Да	Да, при наличии онтологии языка	59%
CogNIAC	Да	Нет	Да	Нет	Нет	Нет данных
Метод опорных векторов	Да	Да	Да	Нет	Да	62%
Деревья принятия решений (ID3)	Да	Да	Да	Нет	Да	52.04%
Деревья принятия решений (Extra Trees)	Да	Да	Да	Нет	Да	66.05%

3.1.2 Выводы

Исходя из результатов сравнения были сделаны следующие выводы:

— большинство рассмотренных методов разрешения анафоры требуют морфологических и синтаксических характеристик;

— большинство рассмотренных методов сосредоточены на разрешении местоименной анафоры;

— среди рассмотренных нет единого метода для разрешения местоименной и кореферентной анафоры;

— методы, основанные на машинном обучении, могут быть применимы к различным языкам при наличии размеченных данных для обучения;

— в разрешении местоименной анафоры методы основанные на машинном обучении показывают большую точность, около 65%.

3.2 Морфологическая разметка

3.2.1 Критерии и сравнение методов

Для сравнения алгоритмов морфологической разметки были выбраны следующие критерии:

— необходимость использования грамматического словаря;

— необходимость контекста словоформы;

— возможность обработки отсутствующих в словаре слов;

— скорость обработки в тысячах слов в секунду;

— возможность применения к различным языкам;

— точность (accuracy, либо PPMV).

Сравнение методов построения морфологической разметки по приведенным критериям представлено в таблице 3.2

Таблица 3.2 — Сравнение методов построения морфологической разметки

Метод	Использование грамматического словаря	Требуется контекст	Обработка не словарных словоформ	Скорость обработки (тыс. слов/с)	Применимость к различным языкам	Accuracy	PPMV
Finite State Transducer	Да	Нет	Нет	Нет данных	Нет	97%	Нет данных
Mystem	Да	Нет	Да	65	Да	Нет данных	778/41
Stemka	Нет	Нет	Да	380	Да	Нет данных	787/493
Pymorphy2	Да	Нет	Да	2	Да	Нет данных	Нет данных
Метод опорных векторов	Нет	Да	Да	Нет данных	Да	92.47%	Нет данных
Meta-BiLSTM	Нет	Да	Да	Нет данных	Да	93.31%	Нет данных

3.2.2 Выводы

Исходя из результатов сравнения были сделаны следующие выводы:

— методам, не основанным на машинном обучении, требуется грамматических словарь для определения морфологических характеристик;

— методам, не основанным на машинном обучении не требуется контекст слоформы для определения морфологических характеристик, однако в случае омонимии, такие методы не дают однозначного разбора;

— методы основанные на конечных автоматах (FST) не могут быть применены к различным языкам;

— большинство рассмотренных методов способны обрабатывать не словарные словоформы;

— алгоритмы стемминга показывают значения скорости обработки в десятки тысяч словоформ в секунду;

— точность (accuracy) рассмотренных методов превышает 90%;

— подход с использованием Meta-BiLSTM показывает наибольшую точность (accuracy) в получении морфологических характеристик среди методов допускающих использование в различных языках.

3.3 Синтаксическая разметка

3.3.1 Критерии и сравнение методов

Для сравнения алгоритмов синтаксической разметки были выбраны следующие критерии:

— необходимость использования морфологических характеристик

— требование проективности языка;

— применимость к различным языкам;

— вычислительная сложность;

— точность.

Сравнение методов построения синтаксической разметки по приведенным критериям представлено в таблице 3.3

Таблица 3.3 — Сравнение методов построения синтаксической разметки

Метод	Использование морфологических характеристик	Требуется проективность языка	Применимость к различным языкам	Вычислительная сложность	Точность
Алгоритм CKY	Нет	Да	Да, при наличии грамматики	$O(n^3)$	Нет данных
Алгоритм shift-reduce	Да	Да	Да, при наличии грамматики	$O(n)$	89%
MaltParser	Да	Нет	Да	$O(n)$	Нет данных
LaPaRus	Да	Да	Нет	Нет данных	81.66%

3.3.2 Выводы

Исходя из результатов сравнения были сделаны следующие выводы:

— большинство методов построения деревьев зависимостей имеют линейную вычислительную сложность;

— большинству рассмотренных методов построения синтаксической разметки требуются морфологические характеристики словоформ;

— фундаментальным ограничением большинства рассмотренных методов является требование проективности языка;

— большинство рассмотренных методов могут быть применены к различным языкам.

4 Сравнение корпусных менеджеров

4.1 Существующие корпусные менеджеры

В таблице 4.1 приведены сведения о параллельных корпусах, содержащих русский язык, обладающих системой правления корпусными данными, позволяющей на различном уровне автоматизации реализовывать рассмотренные виды разметки. В перечисленных корпусах анафорическая разметка не реализована.

4.2 Критерии сравнения корпусных менеджеров

Для сравнения приведенных корпусов и соответствующих систем управления корпусными данными, выделены следующие критерии:

— степень автоматизации построения морфологической разметки;

— степень автоматизации построения синтаксической разметки;

— степень автоматизации выравнивания текстов на различных языках.

Для сравнения степени автоматизации построения разметок и выравнивания, выделены 4 уровня автоматизации:

— 0 — ручное выполнение;

— 1 — ручное выполнение с использованием автоматически предлагаемых вариантов;

— 2 — автоматическое выполнение, с последующей коррекцией и валидацией экспертом;

— 3 — полностью автоматическое построение.

Сравнение систем управления корпусными данными по приведенным критериям представлено в таблице 4.2.

Таблица 4.1 — Сведения о параллельных корпусах, содержащих русский язык

Название корпуса	Доступность	Языки	Морфологическая разметка	Синтаксическая разметка	Автоматизация морфологической разметки	Автоматизация синтаксической разметки
Параллельный корпус НКРЯ [96]	Свободно доступный	33 языка	Реализована	Реализована в двух форматах: Universal Dependencies (UD) и СинТагРус	Автоматическая	Автоматическая для UD; ручная для СинТагРус
Параллельный корпус казахского и русского языков (Национальный корпус казахского языка) [97]	Свободно доступный	Казахский, русский	Реализована	Не реализована	Автоматическая	Не реализована
Татарско-русский параллельный корпус «Туган тел» [98]	Свободно доступный	Татарский, русский	Реализована	Реализована частично	Автоматическая	Автоматическая
Китайско-русский параллельный дискурсивный корпус [99]	Нет данных	Китайский, русский	Нет данных	Реализована на уровне клауз	Нет данных	Автоматическая классификация типов клауз
Параллельный корпус русского языка и диалекта русских циган [100]	Свободно доступный	Русский, диалект русских цыган	Реализована	Не реализована	Автоматическая	Не реализована
Parallel GUM RST Corpus [101]	Свободно доступный	Английский, русский	Реализована	Реализована	Автоматическая	Автоматическая
Параллельный корпус списков славянского паримейника	Ограниченный доступ	Церковнославянский, древнерусский, русский	Реализована	Реализована частично	Автоматическая	Ручная

Таблица 4.2 — Сравнение систем управления корпусными данными параллельных корпусов

Название корпуса	Степень автоматизации построения морфологической разметки	Степень автоматизации построения синтаксической разметки	Степень автоматизации выполнения выравнивания
Параллельный корпус НКРЯ	3	2 (UD) / 0 (СинТагРус)	2
Параллельный корпус казахского и русского языков (Национальный корпус казахского языка)	3	Не реализована	0
Татарско-русский параллельный корпус «Туган тел»	3	2	2
Китайско-русский параллельный дискурсивный корпус	Нет данных	2	0
Параллельный корпус русского языка и диалекта русских цыган	2	Не реализована	1
Parallel GUM RST Corpus	2	2	2
Параллельный корпус списков славянского паримейника	1	1	1

4.3 Выводы

На основании представленного сравнения можно сделать вывод, что наибольшую степень автоматизации выполнения предоставляет системы управления параллельным корпусом НКРЯ и татарско-русским корпусом «Туган тел»; при этом ни одна система управления не обеспечивает полностью автоматизированного выполнения разметок и выравнивания.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Захаров В. П., Богданова С. Ю.* Корпусная лингвистика: учебник для студентов гуманитарных вузов. — Иркутск: ИГЛУ, 2011. — 161 с., ISBN 978-5-88267-316-0.

2. *Грудева Е. В.* Корпусная лингвистика: учеб. пособие. — М: ФЛИНТА, 2012. — 165 с., ISBN 978-5-9765-1497-3.

3. *В. Копотев М.* Введение в корпусную лингвистику. — Прага: Animedia Company, 2014. — 231 с., ISBN: 978-80-7499-067-0.

4. *Доминикан А. И.* Конкорданс как инструмент для изучения значения слова // *Вестник Тверского государственного университета. Серия: Филология*. — 2016. — no. 4. — Pp. 85–89.

5. *Добрынина К. С.* О методике работы над конкордансами // *Филологические науки. Вопросы теории и практики*. — 2012. — Vol. 1, no. 12. — Pp. 59–63.

6. *Калинкин В. М., Шульдишова А. А.* Конкорданс как произведение лексикографии и инструмент исследований (на материале лирической трилогии А. Блока) // *Славистика*. — 2023.

7. *Васильев Н.Л., Жаткин Д.Н.* Конкорданс к поэзии А. С. Пушкина, подготовленный в 1930-х гг. Г. А. Шенгели // *Известия РАН. Серия литературы и языка*. — 2019. — Vol. 78, no. 3. — Pp. 39–51.

8. *Нагель О. В.* Корпусная лингвистика и ее использование в компьютеризированном языковом обучении // *Язык и культура*. — 2008. — no. №4. — Pp. 53–59.

9. *Волошина О. А.* Принципы описания языка в древнеиднийской грамматике Панини // *Вестник нижегородского университета им Н. И. Лобачевского*. — 2018. — no. 1. — Pp. 160–165.

10. The Routledge handbook of corpus linguistics / Ed. by Anne O'Keffe, Michael McCarthy. — Routledge, 2010. — 682 с., ISBN 978-0-203-85694-9.

11. *Даль В. И.* Толковый словарь живого великорусского языка. — Москва : Изд. общ-ва любителей Российской словесности, 1863.

12. *Webster Noah*. An American dictionary of the English language / Ed. by Chauncey A. Goodrich. — Springfield, Mass.: G. & C. Merriam, 1865. — 1768 с.

13. *Johnson Samuel*. A dictionary of the English language / Ed. by Walker John. — London: W. Tegg & co, 1850. — 831 с.

14. *Paul German*. Deutsche Grammatik. — M. Niemeyer, 1916. — 378 с.

15. *Kaeding F. W.* Häufigkeitswörterbuch der deutschen Sprache. — E.S. Mittler & Sohn, 1898. — 671 с.

16. *Jespersen Otto*. A Modern English Grammar on Historical Principles. Part I. Sounds and spellings. — London: George Allen & Unwin, 1909. — 485 с.

17. *Баранов А. Н.* Введение в прикладную лингвистику: Учебное пособие. — М.: Эдиториал УРСС, 2001. — 360 с., ISBN 5-8360-0196-0.

18. *Зубов А.В., Зубова И.И.* Информационные технологии в лингвистике: Учеб. пособие. — М.: Издательский центр «Академия», 2004. — 208 с.

19. *Захаров В. П., Азарова И. В., Митрофанова О. А., Попов А. М., Хохлова М. В.* Моделирование в корпусной лингвистике: специализированные корпусы русского языка / Ed. by Захаров В. П. — СПб.: Изд-во С.-Петерб. ун-та, 2019. — 208 с., ISBN 978-5-288-05902-5.

20. *Leech Geoffrey*. Introducing corpus annotation. — 1997.

21. *Лесников С. В.* Виды разметок текстовых корпусов русского языка // *Научно-техническая информация. Серия 2: Информационные процессы и системы*. — 2019.

22. Национальный корпус русского языка: исследования и разработки. — М., 2019. — Vol. 21 of *Труды Института русского языка им. В.В. Виноградова*.

23. *Богданова-Бегларян Н. В.* Дискурсивная единица типа (того что): функционирование в устной спонтанной речи и возможности лексикографического описания // *Проблемы истории, филологии, культуры*. — 2014. — no. 3 (45). — Pp. 252–254.

24. *Sinclair John M.* Preliminary recommendations on text typology: EAGLES Document EAG-TCWG-TTYP/P: EAGLES (Expert Advisory Group on Language Engineering Standards), 1996. — [Электронный ресурс]. Дата обращения: 19.11.2025. `http://www.ilc.pi.cnr.it/EAGLES96/texttyp/texttyp.html`.

25. Treebanks: Building and Using Parsed Corpora / Ed. by Anne Abeillé. — Dordrecht: Springer, 2003. — Vol. 20 of *Text, Speech and Language Technology*.

26. *Fillmore Charles J.* The case for case // Universals in Linguistic Theory / Ed. by Emmon Bach, Robert T. Harms. — New York, NY: Holt, Rinehart and Winston, 1968. — Pp. 1–88.

27. *Плунгян В. А.* Общая морфология: Введение в проблематику. — 2-е, исправленное изд. — М.: Едиториал УРСС, 2003.

28. *Кибрик А. Е. и др.* Введение в науку о языке / Под ред. О. В. Федорова, С. Г. Татевосов. — М.: Буки Веди, 2019.

29. *Горбань О. А., Косова М. В., Шептухина Е. М.* Структурная разметка деловых документов в диахроническом лингвистическом корпусе: проблемы и решения // *Вестник Волгоградского государственного университета. Серия 2: Языкознание*. — 2021. — Vol. 20, no. 4. — Pp. 5–18.

30. *Загорулько М. Ю., Сидорова Е. А.* Программный инструментарий разработки лингвистических ресурсов // Открытые семантические технологии проектирования интеллектуальных систем (OSTIS-2013): материалы III Междунар. научн.-техн. конф. / Ed. by Голенков В. В. и др. — Минск: БГУИР, 2013. — Pp. 159–164.

31. *Загорулько М. Ю., Кононенко И. С., Сидорова Е. А.* Система семантической разметки корпуса текстов в ограниченной предметной области // Компьютерная лингвистика и интеллектуальные технологии: По материалам ежегодной Международной конференции «Диалог» (2012). Вып. 11(18). Т. 1: Основная программа конференции. — М.: Изд-во РГГУ, 2012. — Pp. 674–683.

32. *Бутенко Ю. И.* Модели и методы автоматической обработки научно-технических текстов в параллельном корпусе: Докторская диссертация / Федеральное государственное автономное образовательное учреждение высшего образования «Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)» (МГТУ им. Н. Э. Баумана). — Москва, 2025. — [Электронный ресурс]. Дата обращения: 30.11.2025. `https://www.frccsc.ru/sites/default/files/docs/ds/002-073-05/diss/47-butenko/ds05-47-butenko_Avtoref.pdf?484&ysclid=mj5lmekm7p401949212`.

33. *Соколов А. В., Батура Т. В.* Разработка системы разрешения анафоры на основе методов машинного обучения // *Программные продукты и системы*. — 2017. — Vol. 30, no. 3. — Pp. 461–468.

34. *Bogdanov A. V., Dzhumaev S. S., Skorinkin D. A., Starostin A. S.* Anaphora analysis based on abbyy compreno linguistic technologies // Компьютерная лингвистика и интеллектуальные технологии: По материалам ежегодной Международной конференции «Диалог». — Vol. 13. — М.: РГГУ, 2014. — Pp. 89–101.

35. *Mitkov Ruslan*. Anaphora Resolution: The State of the Art.

36. *Carter David*. Interpreting Anaphors in Natural Language Texts. Ellis Horwood Series in Artificial Intelligence. — Chichester: Ellis Horwood Limited, 1987.

37. *Baldwin Breck*. CogNIAC: High precision coreference with limited knowledge and linguistic resources // Proceedings of the ACL'97/EACL'97 Workshop on Operational Factors in Practical, Robust Anaphora Resolution. — Madrid, Spain: 1997. — Pp. 38–45. — Association for Computational Linguistics.

38. *Азеркович И. Л.* Использование мер семантической близости для распознавания кореференции в русском языке // *Вестник НГУ. Серия: Лингвистика и межкультурная коммуникация*. — 2019. — Vol. 17, no. 1. — Pp. 65–77.

39. *Ng Vincent*. Machine learning for entity coreference resolution: A retrospective look at two decades of research // AAAI Conference on Artificial Intelligence. — 2017.

40. *Иншакова Е. С., Иомдин Л. Л., Митюшин Л. Г., Сизов В. Г., Фролова Т. И., Цинман Л. Л.* СинТагРус сегодня // Труды Института русского языка им. В.В. Виноградова. Вып. 21. Национальный корпус русского языка: исследования и разработки. — М., 2019. — Pp. 14–40.

41. *Протопопова Е. В., Бодрова А. А., Вольская С. А., Крылова И. В., Чучунков А. С.* Опыт анафорической разметки корпуса и разрешения анафоры с использованием машинного обучения // Компьютерная лингвистика и интеллектуальные технологии: По материалам ежегодной Международной конференции «Диалог». — Vol. 13. — М.: РГГУ, 2014. — Pp. 562–571.

42. *Еремеева Г. Р., Мельникова О. К., Ситтикова Ф. Р.* Разрешение анафоры в вычислительной лингвистике // *Казанский лингвистический журнал*. — 2018. — Vol. 1, no. 1. — Pp. 57–66.

43. *Толпегин П. В., Ветров Д. П., Кропотов Д. А.* Алгоритм автоматизированного разрешения анафоры местоимений третьего лица на основе методов машинного обучения // Компьютерная лингвистика и интеллектуальные технологии: Труды международной конференции «Диалог 2006» / Ed. by Лауфер Н. И., Нариньяни А. С., Селегея В. П. — М.: Изд-во РГГУ, 2006. — Pp. 504–507.

44. *Лукашевич Н. В.* Тезаурусы в задачах информационного поиска. — М.: Изд-во Моск. ун-та, 2011.

45. *Toldova S., Grishina Yu., Ladygina A., Vasilyeva M., Sim G., Azerkovich I.* Russian coreference corpus // Input a Word, Analyze the World: Selected Approaches to Corpus Linguistics / Ed. by Fernando Alonso Almeida, Isabel Ortega Barrera, Elisa Q. Toledo. — Newcastle upon Tyne, UK: Cambridge Scholars Publishing, 2016. — Pp. 107–124.

46. Development and application of a metric to semantic nets / Roy Rada, Hafedh Mili, Ellen Bicknell, Maria Blettner // *IEEE Transactions on Systems, Man and Cybernetics*. — 1989. — Vol. 19, no. 1. — Pp. 17–30.

47. *Wu Zhibiao, Palmer Martha*. Verb semantics and lexical selection // Proceedings of the 32nd Annual Meeting of the Association for Computational Linguistics (ACL-94). — Las Cruces, New Mexico, USA: Association for Computational Linguistics, 1994. — Pp. 133–138.

48. *Leacock Claudia, Chodorow Martin*. Combining local context and WordNet similarity for word sense identification // WordNet: An Electronic Lexical Database / Ed. by Christiane Fellbaum. — Cambridge, MA: MIT Press, 1998. — Pp. 265–283.

49. *Resnik Philip*. Using information content to evaluate semantic similarity in a taxonomy // Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI). — Vol. 1. — Montreal, Quebec, Canada: Morgan Kaufmann, 1995. — Pp. 448–453.

50. *Seco Nuno, Veale Tony, Hayes John*. An intrinsic information content metric for semantic similarity in wordnet // Proceedings of the 16th European Conference on Artificial Intelligence. — Valencia, Spain: IOS Press, 2004. — Pp. 1089–1090.

51. *Lesk Michael*. Automatic sense disambiguation using machine readable dictionaries: How to tell a pine cone from an ice cream cone // Proceedings of the 5th Annual Conference on Systems Documentation. — New York, NY, USA: ACM, 1986. — Pp. 24–26.

52. *Ponzetto Simone Paolo, Strube Michael*. Exploiting semantic role labeling, wordnet and wikipedia for coreference resolution // Proceedings of the Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics. — New York, NY, USA: Association for Computational Linguistics, 2006. — Pp. 192–199.

53. *Zuev K. A., Indenbom M. E., Judina M. V.* Statistical machine translation with linguistic language model // Computational Linguistics and Intellectual Technologies: Proceedings of the International Conference «Dialog 2013». — Vol. 2. — Bekasovo, Russia: 2013. — Pp. 164–172.

54. *Anisimovich K. V., Druzhkin K. Ju., Minlos F. R., Petrova M. A., Selegey V. P., Zuev K. A.* Syntactic and semantic parser based on abbyy compreno linguistic technologies // Computational Linguistics and Intellectual Technologies: Proceedings of the International Conference «Dialog 2012». — Bekasovo, Russia: 2012. — Pp. 90–103.

55. *Winograd Terry*. Understanding Natural Language. — New York: Academic Press, 1972.

56. *Гуренькова О. А., Батура Т. В., Козлова А. А., Свищев А. Н.* Complex approach towards algorithm learning for anaphora resolution in russian language // Компьютерная лингвистика и интеллектуальные технологии: По материалам ежегодной международной конференции «Диалог» (2017). Вып. 16 (23). В 2 т. — М.: Изд-во РГГУ, 2017. — Pp. 89–97.

57. *Quinlan J. Ross*. Induction of decision trees // *Machine Learning*. — 1986. — Vol. 1. — Pp. 81–106.

58. *Yandex LLC*. Tomita parser: Documentation. — 2023. — [Электронный ресурс]. Дата обращения: 28.11.2025. `https://yandex.ru/dev/tomita/`.

59. *Geurts Pierre, Ernst Damien, Wehenkel Louis*. Extremely randomized trees // *Machine Learning*. — 2006. — Vol. 63, no. 1. — Pp. 3–42.

60. *Rani K. Swaroopa*. A machine learning approach for addressing imbalanced data: A systematic review // *Journal of Engineering Sciences*. — 2022. — Vol. 13, no. 8. — Pp. 950–958.

61. *Yandex LLC*. Mystem: Documentation. — 2023. — [Электронный ресурс]. Дата обращения: 28.11.2025. `https://yandex.ru/dev/mystem/`.

62. *Petrov Slav*. Announcing syntaxnet: The world's most accurate parser goes open source. — 2016. — may. — [Электронный ресурс]. Дата обращения: 28.11.2025. `https://research.google/blog/announcing-syntaxnet-the-worlds-most-accurate-parser-goes-open-source/`.

63. *Mikolov Tomas, Sutskever Ilya, Chen Kai et al.* Distributed representations of words and phrases and their compositionality. — 2013.

64. *Комендантов А. С., Матвеев А Г., Светлов А. В.* Автоматизация морфологической разметки архивных документов // *Математическая физика и компьютерное моделирование*. — 2019. — Vol. 22, no. 4. — Pp. 53–62.

65. *Chandía Andrés*. Fst morphological analyser and generator for mapudüngun. — 2022.

66. *Kumar Deepak, Singh Manjeet, Shukla Seema*. Fst based morphological analyzer for hindi language // *International Journal of Computer Science Issues (IJCSI)*. — 2012. — Vol. 9, no. 4. — Pp. 349–353.

67. *Strunk Lonny Alaskuk*. — A Finite-State Morphological Analyzer for Central Alaskan Yup'ik. — Master's thesis, University of Washington, 2020. — [Электронный ресурс]. Дата обращения: 30.11.2025. `https://digital.lib.washington.edu/server/api/core/bitstreams/76ad78b5-6cbc-4d3a-911a-ea313748c378/content`.

68. *Segalovich Ilya*. A fast morphological algorithm with unknown word guessing induced by a dictionary for a web search engine // *Unpublished manuscript*. — 2003. — [Электронный ресурс]. Дата обращения: 30.11.2025. `https://cloudcdn-m9-12.cdn.yandex.net/download.yandex.ru/company/iseg-las-vegas.pdf?lid=7`.

69. *Kovalenko A.* Stemka — Russian and Ukrainian morphological guesser. — 2002. — [Электронный ресурс]. Дата обращения: 10.12.2025. `http://www.keva.ru/stemka/stemka.html`.

70. *Porter Martin F.* The porter stemming algorithm. — 2001. — [Электронный ресурс]. Дата обращения: 10.12.2025. `http://snowball.tartarus.org/algorithms/porter/stemmer.html`.

71. *Pymorphy2 Development Team*. Internals — pymorphy2 0.9.1 documentation. — [Электронный ресурс]. Дата обращения: 20.12.2025. `https://pymorphy2.readthedocs.io/en/stable/internals/index.html`.

72. *Petrochenkov V. V., Kazennikov A. O.* A statistical tagger for morphological tagging of russian language texts // *Automation and Remote Control*. — 2013. — Vol. 74, no. 10. — Pp. 1733–1746.

73. *Schmid Helmut*. SFST Manual / Institute for Natural Language Processing, University of Stuttgart. — [Электронный ресурс]. Дата обращения: 30.11.2025. `https://www.cis.uni-muenchen.de/~schmid/tools/SFST/`.

74. *Smeets Ineke*. A Grammar of Mapuche. — Berlin, Germany and New York, U.S.A.: Mouton de Gruyter, 2008.

75. *Hulden Mans*. Foma: a finite-state compiler and library. — 2009. — 01. — Pp. 29–32.

76. *Зализняк А. А.* Грамматический словарь русского языка. — Москва: Русский язык, 1980.

77. *Porter M. F.* An algorithm for suffix stripping // *Program: electronic library and information systems*. — 1980. — Vol. 14, no. 3. — Pp. 130–137.

78. *Snowball Project*. Russian stemming algorithm. — 2003. — [Электронный ресурс]. Дата обращения: 10.12.2025. `https://snowballstem.org/algorithms/russian/stemmer.html`.

79. *Giménez Jesús, Màrquez Lluís*. SVMtool: A part-of-speech tagger generator based on support vector machines // Proceedings of the Fourth International Conference on Language Resources and Evaluation (LREC 2004). — 2004. — Pp. 43–46.

80. *Институт проблем передачи информации РАН им А. А. Харкевича*. Корпус СинТагРус. — [Электронный ресурс]. Дата обращения: 13.12.2025. `http://proling.iitp.ru/ru/projects/corpus`.

81. *Bohnet Bernd, McDonald Ryan, Simoes Goncalo et al.* Morphosyntactic tagging with a meta-bilstm model over context sensitive token encodings. — 2018.

82. *Dozat Timothy, Qi Peng, Manning Christopher D.* Stanford's graph-based neural dependency parser at the CoNLL 2017 shared task // Proceedings of the CoNLL 2017 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies. — Vancouver, Canada: 2017. — August. — Pp. 20–30.

83. Conll 2017 shared task: Multilingual parsing from raw text to universal dependencies / Daniel Zeman, Martin Popel, Milan Straka et al. — 2017. — 01. — Pp. 1–19.

84. *Закирова Карина, Новикова Ирина, Блюдова Василиса, Кондрашова Екатерина*. Автоматический синтаксический анализ. — НИУ ВШЭ в Нижнем Новгороде: Новости образовательной программы "Фундаментальная и прикладная лингвистика". — 2023. — [Электронный ресурс]. Дата обращения: 15.01.26. `https://nnov.hse.ru/ba/ling/news/808162873.html`.

85. Автоматическая обработка текстов на естественном языке и анализ данных / Е.И. Большакова, К.В. Воронцов, Н.Э. Ефремова и др. — М.: Изд-во НИУ ВШЭ, 2017. — С. 269.

86. *Зобнин А. И., Сахарова А. В.* Универсальная система синтаксической разметки текстов objectate // Компьютерная лингвистика и интеллектуальные технологии / Под ред. А. Е. Кибрик, др. — М.: РГГУ, 2009. — С. 120–126.

87. *Makarov Vladislav*. Cocke–younger–kasami–schwartz–zippel algorithm and relatives. — 2022. `https://arxiv.org/abs/2212.03861`.

88. *Younger Daniel H.* Recognition and parsing of context-free languages in time n^3 // *Information and Control*. — 1967. — Vol. 10, no. 2. — Pp. 189–208.

89. *Jurafsky Daniel, Martin James H.* Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models. — Third draft edition. — 2026. — Draft of January 6, 2026.

90. *Nivre Joakim*. An efficient algorithm for projective dependency parsing // Proceedings of the 7th Conference on Natural Language Learning at HLT-NAACL 2003. — Edmonton, Canada: Association for Computational Linguistics, 2003. — Pp. 1–8. `https://aclanthology.org/W03-3017/`.

91. Maltparser: A language-independent system for data-driven dependency parsing / Joakim Nivre, Johan Hall, Jens Nilsson et al. // *Natural Language Engineering*. — 2007. — Vol. 13, no. 2. — Pp. 95–135.

92. *Covington Michael A.* A fundamental algorithm for dependency parsing // Proceedings of the 39th Annual ACM Southeast Conference. — Athens, Georgia, USA: ACM, 2001. — Pp. 95–102.

93. *Киселёв М. В., Федосеева Д. В.* Синтаксический парсер русского языка LPaRus компании Megaputer Intelligence // Компьютерная лингвистика и интеллектуальные технологии: По материалам ежегодной Международной конференции «Диалог». — Москва: РГГУ, 2017.

94. Compilers: Principles, Techniques, and Tools / Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. — 2nd edition. — Boston, MA, USA: Addison-Wesley, 2007.

95. *Department of Linguistics, Umeå University, Department of Linguistics, Stockholm University*. Stockholm Umeå Corpus. Version 1.0. — Department of Linguistics, Umeå University and Department of Linguistics, Stockholm University, 1997.

96. Национальный корпус русского языка. — `https://ruscorpora.ru/`. — 2026. — [Электронный ресурс]. Дата обращения: 24.01.26.

97. Параллельный подкорпус казахского и русского языков: разработка, функционирование и проблемы / Н.М. Ашимбаева, А.З. Бисенгали, С.К. Кулманов et al. // *Tiltanym*. — 2023. — no. 2. — Pp. 49–61.

98. Русско-татарский (параллельный) подкорпус в общественно-политической и it-областях. — `https://tugantel.tatar/parallel/`. — 2018. — [Электронный ресурс]. Дата обращения: 24.01.26.

99. *Ян И.* Китайско-русский параллельный дискурсивный корпус: выравнивание на уровне клаузы и статистический анализ // *Известия Уральского федерального университета. Серия 2: Гуманитарные науки*. — 2018. — Vol. 20, no. 2 (175). — Pp. 60–73.

100. *Lyashevskaya Olga, Kozhanov Kirill, Arkhangelskiy Timofey*. A parallel corpus for the ruska romani dialect and russian language // Proceedings of the 8th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature. — Association for Computational Linguistics, 2024. — Pp. 1–6.

101. *Chistova Elena*. Bilingual rhetorical structure parsing with large parallel annotations. — 2024. `https://arxiv.org/abs/2409.14969`.